hogeschool vives

IoT 1

R.Buysschaert

GR Handelswetenschappen, bedrijfskunde

PBA toegepaste informatica

GS Artificial Intelligence

Fase 1, 2, 3

V5R313, V5R433

8138

HWB226

Het digitaal cursusmateriaal is enkel bestemd voor persoonlijk gebruik door de gebruiker. Het is de gebruiker niet toegestaan om de digitale cursus geheel of gedeeltelijk te kopiëren, verkopen, verdelen, doorgeven, vertalen, verspreiden, weergeven, reproduceren, publiceren, verhuren, leasen, onderlicentiëren, vergunnen met licentie of op andere wijze over te dragen.

r1065698 Tim Kindt

r1065698 Tim Kindt

Theorie IoT 1

Academiejaar: 2025 – 2026

Auteur: R.Buysschaert

Wat is een microcontroller?

Inleiding

Microcontroller of microprocessor? Velen kunnen niet onmiddellijk het verschil aanduiden, hoewel dit niet zo moeilijk is. **Microprocessoren** bevatten quasi nooit een intern programma- of datageheugen (met uitzondering van het cache geheugen en de interne registers) en dus beschikken ze altijd over een adres- en databus die extern beschikbaar is. Soms is deze bus gemultiplexed: daarbij is de bus afwisselend adresbus of databus. De toestand wordt bepaald aan de hand van een extra controlelijn.

Microcontrollers hebben wel intern geheugen en meestal geen externe adres- en databus. Naast dit interne geheugen beschikken microcontrollers ook over een aantal extra voorzieningen, zoals seriële interfaces (UART, SPI, I^2C, …), telleringangen, analoog-digitaal en digitaal-analoog omzetters, interne klok(ken), USB controller, CAN bus controller, Ethernet, …

Er zijn **verschillende populaire microcontrollerreeksen**. Denk maar aan de Microchip **PIC** microcontrollers, Atmel **AVR**-reeks, **ARM** microcontrollers, …

Microchip en Atmel zijn heel populair, zowel in het hobbymilieu als in de industrie. Dit heeft onder andere te maken met de kostprijs van de controllers, maar ook dankzij de gratis beschikbare ontwikkelomgeving en de ruime ondersteuning via de websites van de producenten. Atmel levert ook de microcontrollers voor op de **Arduino**-bordjes. Die Arduino-bordjes worden veel gebruikt bij mensen die de instap naar microcontrollersystemen willen maken. In de industrie wordt Arduino (heel) weinig gebruikt.

In deze cursus IoT Devices, maken we gebruik van het **NUCLEO-F091RC** bord. Dat is een low-cost bord met daarop een **32-bit ARM microcontroller**: STM32F091RC. Dat is een “Entry-level **ARM Cortex-M0** Access line MCU with 256 Kbytes Flash running on 48 MHz”.

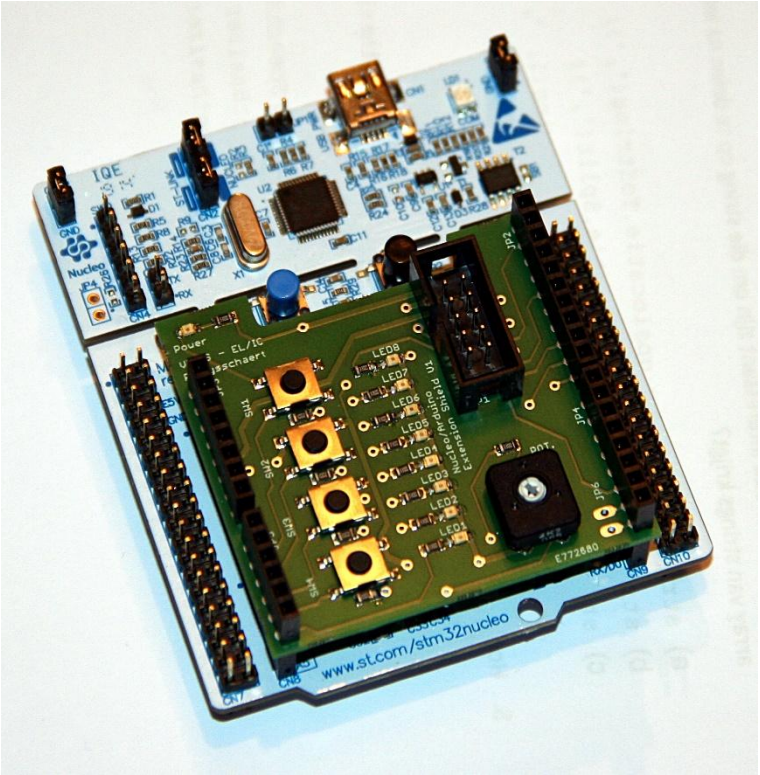
Bovenop dat Nucleo-bord, pluggen we een zelf gemaakt **extension shield**. Dat shield zorgt voor wat invoer- uitvoermogelijkheden (4 knoppen, 8 LED’s, UART, SPI, I^2C, …). Later koppelen we dan ook andere modules en sensoren, zodat een breed scala aan oefeningen mogelijk wordt.

De **schema’s** van zowel het NUCLEO-F091RC bord als van het Nucleo Extension Shield, zijn documenten die je steeds bij je moet hebben tijdens de lessen Microcontrollers. Liefst op papier, anders in PDF op je laptop.

Afbeeldingen van de bordjes vind je op de volgende pagina. De inhoudelijke bespreking volgt verder in de cursus.

r1065698 Tim Kindt

Figuur 1: Bestukken van de Nucleo Extension Shields



Figuur 2: Nucleo Extension Shield op het NUCLEO-F091RC bord

r1065698 Tim Kindt

Voorkennis

Om dit vak te kunnen volgen is *strikt gezien* geen speciale voorkennis vereist. Het is echter wel zo dat wat ervaring met andere microcontrollersystemen, **C-taal en algemene programmeerkennis een groot pluspunt** zijn. Ook **binair rekenen en digitale basislogica** zijn heel belangrijk. Heb je die ervaring niet of onvoldoende, dan is het essentieel dat de opgedane kennis continu en **overvloedig geoefend wordt** tijdens het volledige academiejaar/semester.

Er is echter één groot voordeel. **Microcontrollers zijn de max ;-)** Ze laten je toe een wild idee om te zetten naar een werkend geheel!

Design your future!

r1065698 Tim Kindt

Onderdelen van een microcontroller

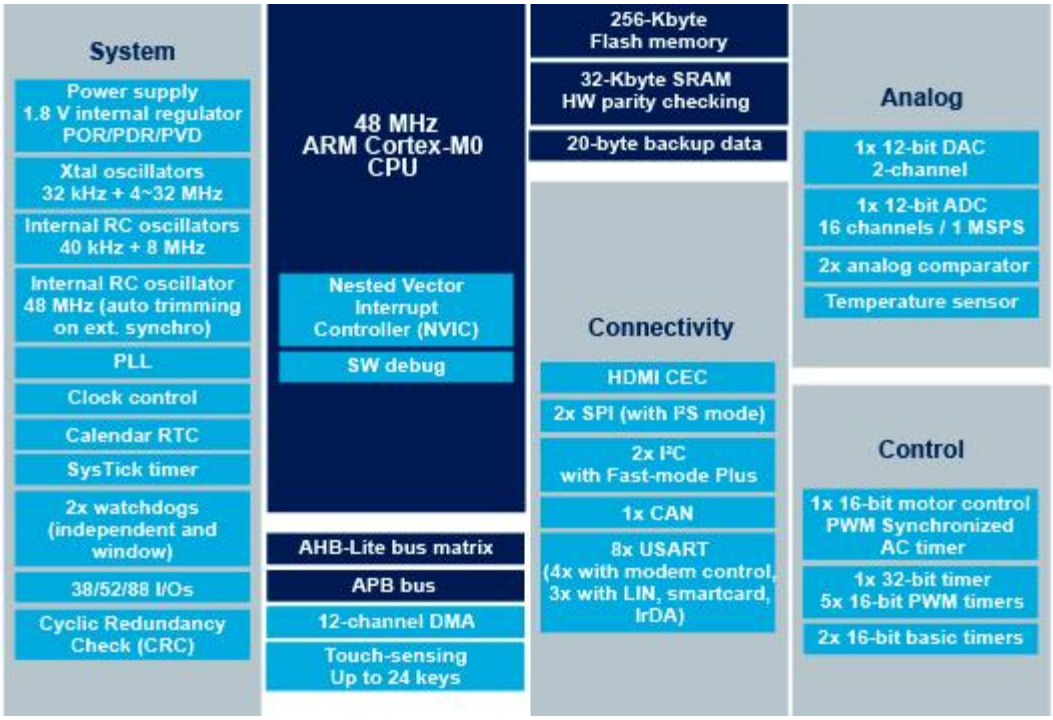
Veel voorkomende onderdelen bij een microcontroller zijn:

- **CPU** (Central Processing Unit): voert de instructies uit (logische, aritmetische, lees/schrijf acties).
- **Oscillator**: maakt met behulp van een kristal een klok waarop de logica werkt. Hier wordt dus de 'kloksnelheid' van heel het systeem bepaald.
- **Bussen**: adres-, data- en controlebussen die de verschillende blokken verbinden.
- **ROM** (Read Only Memory): geheugen dat niet volatiel is. Dit geheugen wordt gebruikt om het 'programma' in op te slaan.
- **RAM** (Random Access Memory): geheugen dat volatiel is, maar sneller dan andere types geheugen (ROM).
- **Timers**: die geven signalen wanneer een bepaalde tijd verstreken is.
- **Interrupt** controller: die bundelt verschillende interrupts die al dan niet doorgegeven worden aan de processor.
- **IO** (input-output):
    - Parallelle IO: één byte wordt bijvoorbeeld met 8 lijnen doorgestuurd naar een ontvanger.
    - Seriële IO: één byte wordt bit per bit over 1 lijn overgestuurd (voorbeeld USB en de seriële poort).
    - Analoge IO: Het voltage toegevoerd aan een analoge input wordt door een ADC(Analog Digital Converter) omgezet tot een binaire representatie. Bij een analoge output wordt een voltage gegenereerd uit een digitale waarde. Dit wordt bijvoorbeeld gebruikt bij het genereren van de voltages voor een luidspreker.
- **Watchdog** timer: Dit is een component die af en toe een trigger moet krijgen, zo niet wordt de CPU gereset. Dit voorkomt dat de CPU, door bijvoorbeeld een bug, voor altijd in een lus blijft 'vastzitten'.

r1065698 Tim Kindt

Bovenstaande lijst is niet compleet. De fabrikant van een microcontroller kan ervoor kiezen om meer of minder van dergelijke (logische) blokken te implementeren. Het is aan de ontwerper van het volledige systeem om een verstandige keuze te maken qua microcontrollertype.

Om een duidelijk overzicht te bekomen van de interne modules van onze ARM 32 µC, bekijk je best onderstaand blokschema:



Figuur 3: overzicht van de modules in de STM32F091RC (bron: st.com)

r1065698 Tim Kindt

Voorbeelden van dagelijks gebruik

Microcontrollers zijn niet meer weg te denken uit heel veel **hedendaagse toestellen**. Enkel voorbeelden hieronder…

Thuis:

- Centrale verwarming en bijhorende thermostaat (temperatuur, maar ook LCD, communicatie, …).
- Inbraakbeveiliging
- Microgolfoven
- Speelgoed
- Uurwerk
- Vaatwasser
- Audioversterker
- Domoticasystemen
- Batterijladers
- Smartphone
- …

Kantoor:

- Printer
- Scanner
- e-ID-lezer
- Toetsenbord
- Gebouwencontrole (RFID)
- Telefoontoestellen en –centrale
- Tablet PC
- …

Auto:

- Klimaatcontrole
- Parkeerassistentie
- Remassistentie
- GPS
- Alle systemen aangesloten op de CAN-bus
- …

Kortom: quasi alle toestellen die wat 'intelligentie' bezitten, hebben meestal één of andere vorm van microcontroller in zich…

Wil je zelf aan de slag met enkele 'Do-It-Yourself'-projectjes? Dan kan je online heel veel voorbeelden vinden. Zo ook op: www.rubu.be ;-)

r1065698 Tim Kindt

Indelen van microcontrollers

Microcontrollers kunnen onder meer ingedeeld worden volgens:

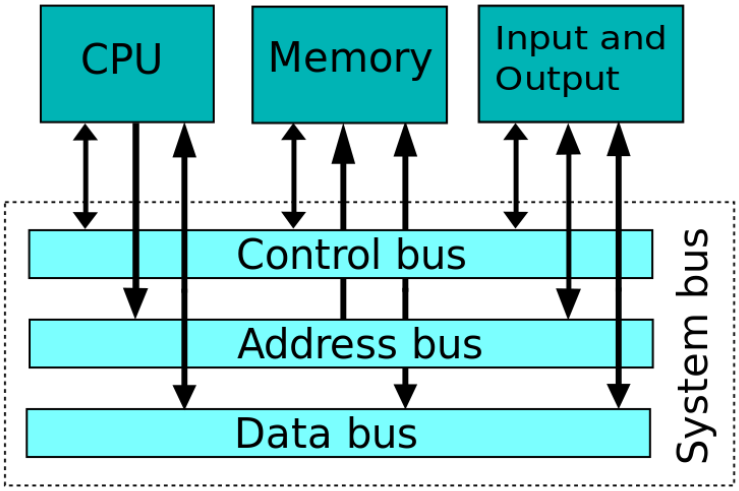
- Architectuur
- Woordbreedte
- Complexiteit van instructieset RISC/CISC
- Type van de instructieset
- Fabrikant

r1065698 Tim Kindt

Architectuur

Von Neuman architectuur

Bij een **Von Neuman** architectuur is er slechts één systeembus. De meeste **microprocessoren** zijn hoofdzakelijk volgens deze architectuur gemaakt. Zowel de gegevens uit het datageheugen als uit het programmageheugen moeten **beide over dezelfde bussen** getransporteerd worden. Het grootste voordeel van dergelijk ontwerp is de eenvoud van het systeem. Het nadeel, de bussen worden gedeeld…

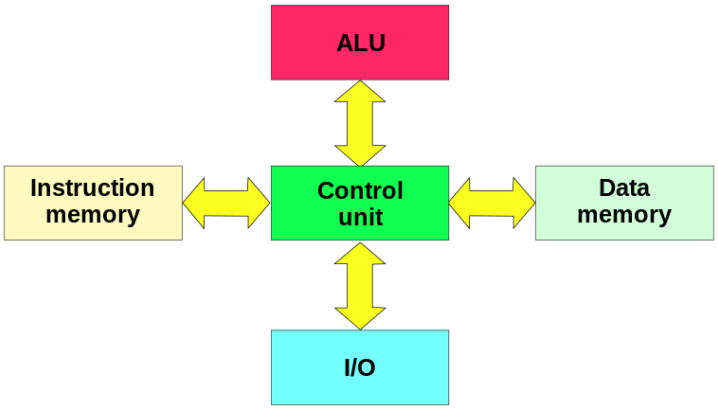


Figuur 4: Von Neuman architecture (bron: wikipedia.org)

r1065698 Tim Kindt

Harvard architectuur

Bij een **Harvard** architectuur zijn het **datageheugen en het programmageheugen elk apart verbonden** via een bus met de CPU. Het voordeel is dus dat deze twee geheugens hun eigen bus hebben en dus weinig of geen rekening moeten houden met elkaar. De meeste **microcontrollers** zijn met deze architectuur ontworpen.



Figuur 5: Harvard architecure (bron: wikipedia.org)

r1065698 Tim Kindt

Een ander voordeel van de Harvard architectuur is dat het programma- en datageheugen verschillende eigenschappen kunnen hebben. Bijvoorbeeld een andere timing of woordbreedte. Hierbij is het ook mogelijk om ROM te gebruiken voor het instructiegeheugen en RAM voor het datageheugen.

Bij een ***gewijzigde Harvard architectuur*** (modified Harvard architecture) zijn het instructie- en datageheugen **niet volledig gescheiden**. Dit komt voor bij de moderne processoren die een aparte cache hebben voor instructie- en datageheugen maar na de cache worden beide bussen terug op één bus aangesloten.

Er zijn ook microcontrollers die geen volledige scheiding hebben om bijvoorbeeld strings in ROM te kunnen laten staan en niet naar RAM te moeten kopiëren. Bij de PIC18-reeks is het bijvoorbeeld zo dat de strings van het programma niet naar RAM worden gekopieerd en in C via een speciale pointer kunnen uitgelezen worden.

Woordbreedte

De woordbreedte bepaalt op hoeveel data de processor per klokpuls bewerkingen uitvoert.

Microprocessoren uit laptops en computers zijn tegenwoordig bijna allemaal 64-bit. Zelfs voor de kleine pc's zoals een netbook wordt veelal een 64 bit processor gebruikt.

Er zijn momenteel nog veel 8-bit en 16-bit microcontrollers in gebruik. Er is wel duidelijk een trend naar 32 bit microcontrollers. De prijs van dergelijke 32-bitters is de laatste jaren fel gedaald, waardoor het ook in goedkoper systemen mogelijk wordt om een 32-bit microcontroller te gebruiken.

In de praktijk worden de meeste microcontrollers geprogrammeerd in een programmeertaal genaamd: C-taal. In die programmeertaal C, hebben de **integer types niet altijd dezelfde grootte (breedte)**. Dit stelt natuurlijk problemen bij gebruik van deze integer types. Om deze problemen te vermijden definieert men types die onafhankelijk van de woordbreedte dezelfde grootte hebben.

In ons geval wordt dit bijvoorbeeld:

```
/* exact-width signed integer types */
typedef   signed          char int8_t;
typedef   signed short     int int16_t;
typedef   signed           int int32_t;
typedef   signed       __INT64 int64_t;

/* exact-width unsigned integer types */
typedef unsigned          char uint8_t;
typedef unsigned short     int uint16_t;
typedef unsigned           int uint32_t;
typedef unsigned       __INT64 uint64_t;
```

Figuur 6: exact-width integers (stdint.h)

Als we dus bijvoorbeeld een getal willen voorstellen dat als bereik 0 tot 54205 moet hebben en toch zo efficiënt mogelijk gebruik moet maken van de beschikbare 'resources' , kunnen we het beste het type *uint16_t* gebruiken…

Bekijk zelf eens enkele andere mogelijkheden.

r1065698 Tim Kindt

Complexiteit van de instructieset

Complex Instruction Set Computer (CISC)

Eén enkele instructie voert verschillende acties uit. Bijvoorbeeld: een waarde uit het geheugen lezen, er een bewerking op doen en wegschrijven naar het geheugen.

Reduced Instruction Set Computer (RISC)

De instructies zijn eenvoudig zodat een hoge kloksnelheid kan gehaald worden. De term “reduced” slaat dus op het werk dat gedaan wordt binnen één instructie en niet op het aantal verschillende commando’s dat de instructieset telt.

r1065698 Tim Kindt

Type van de instructieset

De bekendste instructieset is wellicht de **x86**. Dit is een instructieset die ontwikkeld werd voor het gebruik op de Intel 8086 processoren. Oorspronkelijk was dit een **16**-bit instructieset. Later volgden 32 en 64-bit versies.

Andere instructiesets zijn PowerPC, SPARC, PIC, ARM.

Veelal is een instructieset gekoppeld met een fabrikant. Echter de x86 wordt door meerdere fabrikanten gebruikt.

Van de ARM instructieset zijn er ook meerdere fabrikanten. De firma ARM produceert zelf geen microprocessoren maar verkoopt ontwerpen aan andere firma's. Ook van onze microcontroller STM32F091RC is de architectuur ontworpen door het bedrijf ARM (www.arm.com), maar in een chip geplaatst door de fabrikant ST (www.st.com).

r1065698 Tim Kindt

Fabrikant

Dat er vele fabrikanten zijn van microcontrollers, dat had je wel al door. Een klein overzicht vind je hieronder:

ST Microelectronics

In deze cursus gaan we het vooral hebben over de STM32 reeks van STMicroelectronics.

STMicroelectronics heeft twee reeksen microcontrollers, de STM8 (8 bit) en de STM32. Beide reeksen zijn gebaseerd op de ARM Cortex processor cores. Door de relatief lage prijs en grote rekenkracht, winnen de 32-bit ARM microcontrollers steeds meer aan populariteit. In de industrie is dit reeds een geruime tijd quasi de standaard…

Microchip

Microchip is een grote speler en heeft 2 grote reeksen, de PIC18 en PIC32 microcontroller reeks. De PIC32 reeks is gebaseerd op de MIPS M-Class of M4K core. De PIC18 is een 8 bit microcontroller. Zeer populair bij de hobbyist en de KMO.

Bij het schrijven van dit cursusmateriaal, is recentelijk bekend gemaakt dat **Microchip** de 'concullega' **Atmel** zal overnemen. Microchip telt iets meer dan 3 miljard euro neer voor het opkopen van Atmel. In een brief naar de klanten wordt beloofd om het productengamma niet onmiddellijk 'overhoop' te gooien… Een grote fusie van twee heel belangrijke spelers.

r1065698 Tim Kindt

Freescale

Freescale heeft een Coldfire, Power Architecture en Kinetis-ARM reeks microcontrollers.

NXP

NXP is de zelfstandig geworden semiconductor afdeling van Philips en heeft nu een hele reeks microcontrollers op basis van verschillende ARM cores.

Texas Instruments

Texas Instruments heeft een viertal reeksen microcontrollers. Naast twee reeksen die gebouwd zijn rond ARM processoren is er nog de MSP430-16bit en C2000-32bit reeks.

Renesas

Renesas is een consortium van NEC, Hitachi en Mitsubishi. Renesas beschikt over een reeks ultra-low-power 8 en 16 bit microcontrollers (RL78, 78K en R8C) en een 32bit RISC (V850, SuperH) en CISC (RX) reeks.

Infineon

Infineon heeft een reeks van 8 bit (XC800), 16 bit (XC2200) en 32 bit (TriCore) microcontrollers.

Silicon labs

Silicon labs heeft een reeks gemaakt met een gepijplijnde versie van de 8081 (8 bit) microcontroller.

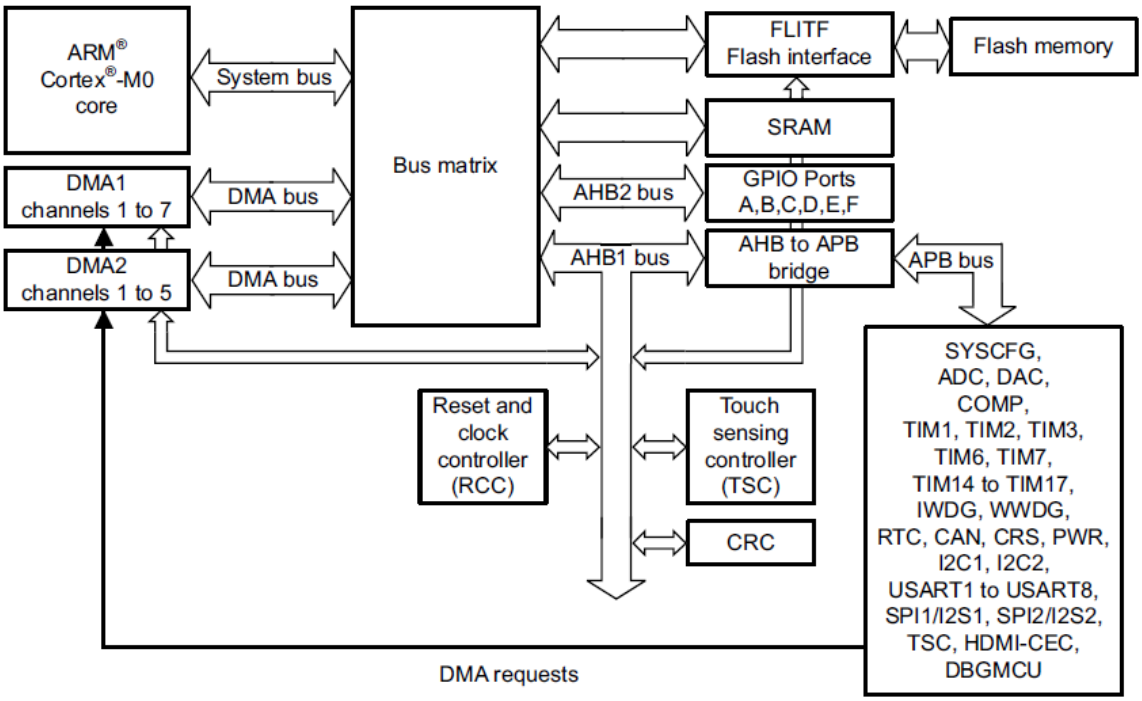
Bovenstaande lijst is absoluut niet volledig en gevoelig voor verandering. Fabrikanten maken nieuwe reeksen, fusioneren, verlaten een gamma, … De meest recente info is terug te vinden op de respectievelijke website.

r1065698 Tim Kindt

Ons STM32-architectuur (STM32F091RC Cortex M0)

Blokschematisch

Onze STM32F091RC ARM microcontroller heeft onderstaande **architectuuropbouw**:



Figuur 7: STM32F0-architectuur (bron: STM32F0 Reference Manual - DM00031936)

Merk op dat het **niet meer zo eenvoudig is om hier nog de opsplitsing te maken tussen Von Neuman en Harvard**. De ARM-architecturen zijn eigenlijk **specifiek ontworpen** met meestal één bepaalde focus voor ogen.

In ons geval is dit bij de Cortex M0-reeks: *"The ARM® Cortex®-M0 processor is the smallest ARM processor available. The exceptionally **small silicon area**, **low power** and minimal code footprint of the processor enables developers to achieve **32-bit performance at an 8-bit price point**, bypassing the step to 16-bit devices. The ultra-low gate count of the Cortex-M0 processor also enables its deployment in analog and mixed signal devices."*

(bron: http://www.arm.com/products/processors/cortex-m/cortex-m0.php)

Omschrijving

Masters

Bovenstaand blokschema heeft eigenlijk faciliteiten voor 3 'masters'. Een 'master' speelt 'baas' in een elektronisch systeem. Andere componenten uit het zelfde systeem moeten gehoorzamen/luisteren naar wat de master hen vraagt. **De drie masters zijn: ARM Cortex M0 core, DMA1 en DMA2**.

In de STM32F091RC is hét hart van het systeem (de belangrijkste master), de ARM Cortex M0 core. Dit is de CPU waar alle berekeningen plaatsvinden. Een Cortex M0 core is een 'entry level' ARM

r1065698 Tim Kindt

microcontroller. Dat wil zeggen dat het een van de eenvoudigste ARM Cortex cores is die er bestaan. De complexere Cortex cores worden bijvoorbeeld ingezet bij smartphones maar vallen buiten het bestek van deze (inleidings-)cursus.

De DMA1 en DMA2 module zorg voor **Direct Memory Access faciliteiten**. DMA –gebruik is echter niet voor beginners, daarom volgt meer uitleg aan het einde van deze module.

System bus

De system bus uit bovenstaand blokschema, verbindt de CPU met de Bus Matrix. Die **Bus Matrix regelt de arbitrage tussen de core en de DMA bussen**. Het is dus die Bus Matrix die bij een DMA request de nodige stappen zal ondernemen om de CPU te 'ontheven' van zijn heel repetitieve tijdrovende acties en zo de efficiëntie van heel het systeem omhoog te krikken door via DMA-technieken bepaalde acties te voltooien.

Slaves

Als er masters zijn, moeten er ook slaves zijn. In de Cortex M0 core zijn dit er vier:

- **FLITF**: FLash memory InTerFace
- **SRAM**: Static RAM
- **AHB1** met zijn AHB naar **APB** (Advanced Peripheral Bus) brug: Advanced High-performance Bus 1
- AHB: Advanced High-performance Bus 2

De arbitrage van alle bussen (zorgen dat er geen bits botsen en dergelijke), gebeurt in de Bus Matrix.

AHB naar APB brug

De Advanced High-performance Bus 1, bezit een 'brug' (bridge) naar de Advanced Peripheral Bus. Die laatste, de APB, maakt connectie met bijvoorbeeld: Timers, UART's, RTC, I^2C, … Dus: doorheen die brug, kunnen de peripherals (randcomponenten) de CPU bereiken. Dit wordt verder in de cursus heel belangrijk.

r1065698 Tim Kindt

Arduino

Wat is een Arduino?

Wie kan dat beter uitleggen dan de bedenker?

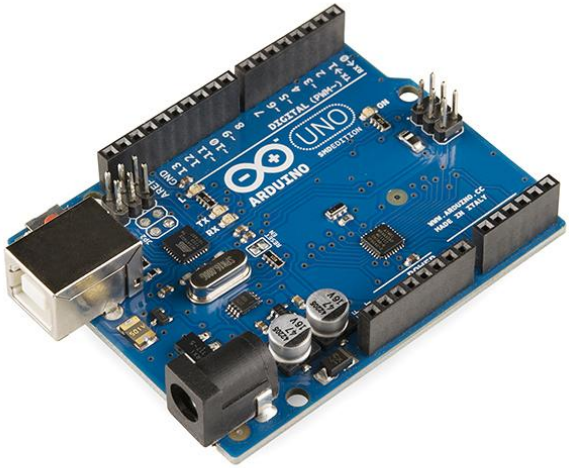
Vandaar: "*Arduino is an **open-source prototyping platform** based on easy-to-use hardware and software.*

…

*Arduino was born at the Ivrea Interaction Design Institute as **an easy tool for fast prototyping, aimed at students without a background in electronics and programming**. As soon as it reached a wider community, the Arduino board started changing to adapt to new needs and challenges, differentiating its offer from simple 8-bit boards to products for IoT applications, wearable, 3D printing, and embedded environments. All Arduino boards are completely open-source, empowering users to build them independently and eventually adapt them to their particular needs. The software, too, is open-source, and it is growing through the contributions of users worldwide.*" (bron: arduino.cc)

In feite is een Arduino bord een 'gewoon' microcontroller bord met daarop een **Atmel microcontroller** en een **Arduino bootloader**. Die bootloader is een klein stukje firmware die ervoor zorgt dat je de microcontroller kan herprogrammeren via de UART (seriële poort). Op die manier heb je geen extra programmer nodig en kan de beginnende programmeur/elektronicus heel snel met weinig achtergrondinfo zijn/haar systeem aan de praat krijgen.

Het meest gebruikte Arduino bord, de Arduino Uno R3, heeft on board een ATmega328p microcontroller. Dat is een 8-bit microcontroller die op 16MHz werkt.



Figuur 8: Arduino Uno R3 (bron: wikipedia.org)

r1065698 Tim Kindt

Voordelen van bovenstaand bord zijn:

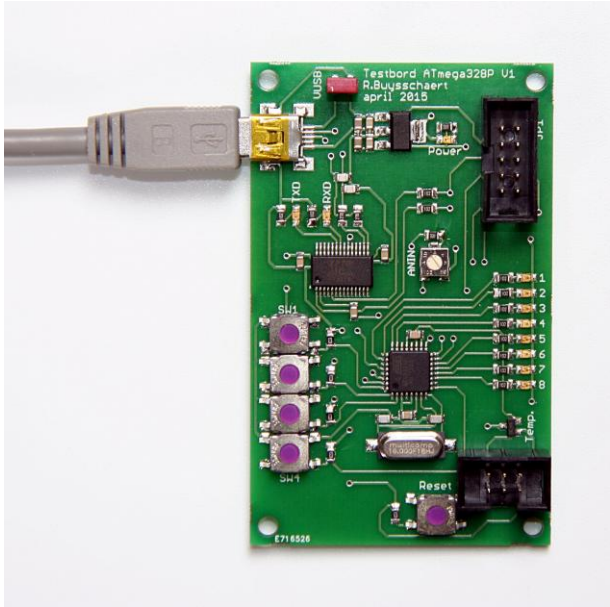
- Prijs
- (stacking) headers voorzien zodat je snel kan prototypen
- Programmeerbaar via USB (geen externe programmer nodig)
- Veel shields beschikbaar

Nadelen:

- Weinig (tot geen) direct beschikbare IO (knoppen en LED's). Vooraleer je kan starten met programmeren moet je dus eerst hardware koppelen.
- De print heeft een vaste afmeting, die misschien absoluut niet past in het project dat je voor ogen hebt.
- De standaard IDE van Arduino is heel eenvoudig, maar mist enkele handige tools die een professional er graag bij heeft (vooral qua debugging)…
- Moeilijk te gebruiken voor tijd kritische systemen (Timers, vaste looptijden, …)

Binnen de opleiding Elektronica-ICT aan de Katholieke Hogeschool VIVES zijn er Arduino bordjes beschikbaar die on board over wat IO beschikken. De "**Testbord ATmega328p**" systemen.

Meer info: https://rubu.be/index.php?request=testbordatmega328p&category=diy



Figuur 9: Testbord ATmega328p

r1065698 Tim Kindt

Waarom we geen Arduino gebruiken

Arduino is leuk, gemakkelijk, je krijgt snel 'leven' in je projecten. Maar je kan het **niet gebruiken als 'standaard** om embedded systems te leren programmeren'.

Op een grondige manier microcontrollers leren programmeren is niet evident. Als je het echt goed wil doen, moet je tijd investeren en ervaring opdoen. Alleen op die manier kan je de **gelimiteerde resources van een embedded system ten volle benutten**.

"**Using an Arduino to learn programming is like using MacDonalds to learn cooking; you get your meal, very fast, but you don't get the skills to cook yourself.** When you need a quick meal, the Mac can be OK (debatable, but just to make my point), but it's not a cooking class." (bron: http://www.hackvandedam.nl/blog/?p=762)

Andere vaak gehoorde negatieve bevindingen:

- Slechte IDE
- Opsplitsen van code wordt niet gestimuleerd in de IDE
- Slecht gekozen benamingen: WriteAnalog() bijvoorbeeld. Zet eigenlijk geen analoge waarde, maar stelt een PWM-signaal in… Daarenboven kan je niet zomaar de PWM-frequentie aanpassen!
- Geen exact-width variabelen
- Documentatie laat teveel zaken onbesproken voor de professional. Dat kan handig zijn voor de beginner, maar eenmaal je 'meer' wil kom je een 'black box' tegen.
- Door het black box-principe is het heel moeilijk om firmware te schrijven met een minimale footprint.

r1065698 Tim Kindt

Wees dus op je hoede voor het Arduino beest ;-). Het heeft zeker zijn plaats binnen de elektronica-ICT-wereld, maar niet overal!

"The Arduino is great 'programming fastfood': easily available, quick result and sometimes quite tasty. But **it isn't suitable for educating 'how to program**' or '**getting performance out of microcontrollers**', or use it as a first step that has an advanced level for enthousiasts." (bron: http://www.hackvandedam.nl/blog/?p=762)

Programmeertalen

Assembly

Assembly is een ‘low-level’ programmeertaal. Assembly staat dus heel dicht bij de hardware.

Meestal is het zo dat een assembly instructie een 1 op 1 overeenkomst heeft met de effectief mogelijk uit te voeren instructies van een computer/microcontroller.

Een voorbeeld van een assembly programma vind je hieronder:

```
main
    CALL  init

    BCF   0x03,RP0
    BCF   0x03,RP1
    CLRF  teller
    CLRF  teller+1

    CLRF  adResult
    CLRF  dutyCycle

m001
    CALL  getAdResult
    BCF   0x03,RP0
    BCF   0x03,RP1
    MOVWF adResult

    MOVLW 102
    SUBWF adResult,W
    BTFSC 0x03,Carry
    GOTO  m002

    BCF   0x03,Carry
    RRF   adResult,W
    MOVWF dutyCycle
```

Figuur 10: assembly code snippet (voor PIC16F88)

Hierin zie je enkele ‘assembly instructies’, zoals:

MOVLW = MOV Literal to Work register

BCF = Bit Clear File

…

r1065698 Tim Kindt

Voordeel hiervan is dat je exact weet **hoeveel tijd** je programma in beslag neemt. Iedere instructie heeft namelijk een duidelijke omschrijving van zijn duurtijd (meestal 1 of 2 klokcycli). Op die manier kan je heel tijd kritische programma's ontwerpen.

Het **nadeel** is vooral de moeilijke **leesbaarheid**. De afkortingen van de instructies zijn niet zomaar als tekst af te lezen en te begrijpen. Je kan je code ook minder 'mooi' opdelen in logische blokken. Als je het slecht aanpakt heb je heel snel 'spaghetti code'.

Assembly was 'in den beginne' de enige manier om microcontrollers te programmeren. Nu wordt ze minder toegepast, tenzij in enkele heel speciale (tijd kritische) gevallen.

C

C-taal is de **meest gebruikte taal** heden ten dage voor het programmeren van embedded systems. Het is ontworpen om relatief ver van de hardware (vergelijken met assembly althans), de programmeur heel flexibel code te laten schrijven. Het is een programmeertaal die op vele plaatsen inzetbaar is en 'sturctured programming' (structureel programmeren) ondersteunt. De C-code kan via een compiler omgezet worden naar machinetaal voor bijvoorbeeld een microcontroller.

Kernighan en Ritchie schreven de originele C-definitie in het boek: "The C Programming Language".

Op een goede manier C-taal schrijven is echter niet zo evident. Om jezelf de basistechnieken aan te leren zijn er vele goede boeken en tutorials. Eén veel gebruikte is: "**Essential C**", door Nick Parlante.

Dit document kan je ook online terugvinden (onder andere Toledo) en maakt deel uit van de cursus. Ten gepaste tijde zal teruggegrepen worden naar dit document om op die manier een goede C-programmeertechniek aan te leren.

r1065698 Tim Kindt

C++

C++ is als het ware een 'opvolger' van C en is ook algemeen inzetbaar maar met ondersteuning voor object georiënteerd programmeren. C++ staat dus iets 'verder' van de hardware maar is ook goed uitgerust om systemen met gelimiteerde resources te gaan programmeren. C++ wordt dus ook dikwijls gebruikt als er microcontrollers geprogrammeerd moeten worden.

C++ heeft ook veel invloed uitgeoefend op het ontwikkelen van ander programmeertalen zoals C# en Java.

C#

C# is oorspronkelijk een Microsoft .NET programmeertaal die nog verder dan assembly, C en C++ van de hardware staat. Naarmate embedded systems alsmaar krachtiger gekomen zijn, werd het ook mogelijk om een microcontroller in C# te gaan programmeren. Het is echter zo dat de microcontroller niet direct de C#-instructies kan verwerken. Er is een compiler en een framework nodig om alle draaiende te krijgen. Een voorbeeld framework is: .NET Micro Framework.

Op dit moment lijkt het echter niet zo dat die manier van werken een hoge vlucht zal nemen. Maar je weet echter nooit…

Figuur 11: .NET Micro Framework logo (netmf.com)

r1065698 Tim Kindt

‘Onze’ STM32F091RC

Waarom?

De keuze van de microcontroller bij de aanvang van een embedded project, is heel belangrijk. Zaken waar je zeker rekening moet mee houden zijn:

- Wat mag de kostprijs zijn?
- Hoeveel bordruimte (grootte) is er beschikbaar op de print?
- Hoeveel berekeningen per seconde wil je gebruiken?
- Welk stroomverbruik is toegelaten?
- Op welke spanning wil je werken?
- Hoeveel IO’s heb je nodig?
- Welke communicatiekanalen wil je gebruiken (UART, SPI, CAN, …)?
- In welke programmeertaal wil je programmeren?
- Hoe vlot kan je de microcontrollers aankopen?
- …

Met bovenstaande in het achterhoofd is het dus **onmogelijk om de ‘perfecte’ microcontroller voor te stellen**. In deze module (Microcontrollers 1) is gekozen voor het gebruik van de STM32F091RC microcontroller om volgende redenen:

- 32-bit microcontroller (grote rekenkracht)
- Tot 48MHz klokfrequentie (grote rekenkracht)
- Snel beschikbaar via de Nucleo borden (NUCLEO-F091RC)
- De Nucleo borden zijn uitbreidbaar via Arduino compatibele stacking headers en bevatten een onboard ST-link programmer.
- Alle basis communicatiekanalen zijn aanwezig (UART, SPI, I²C, CAN, …)
- Prijs
- Het is een ARM microcontroller en wordt wellicht ‘de’ standaard de komende jaren.
- …

r1065698 Tim Kindt

Specificaties

Hieronder een korte opsomming van de meest essentiële specificaties:

- ARM gebaseerd Cortex M0 core
- 32-bit systeem
- 256KB Flash geheugen (voor opslag van je firmware)
- 32KB SRAM
- Tot 48MHz (optie tot gebruik van interne klok)
- 3V3 voedingsspanning, met enkele 5V tolerante IO’s
- DMA mogelijkheden
- 12-bit AD converter
- 12 timers
- UART, SPI, I²C en CAN
- HDMI CEC
- Beschikbaar in LQFP64, dus zelf te solderen… ☺

NUCLEO-F091RC

Schema

Het schema van het NUCLEO-F091RC bord is gemeenschappelijk met andere Nucleo-borden waarop een 64-pin ARM op gesoldeerd is. Dit geeft onmiddellijk ook aan dat STMicroelectronics de kaart trekt van 'opschaalbaarheid' en 'compatibiliteit'. Als je een systeem hardwarematig ontworpen hebt, de print getekend dus, kan je in principe een andere 64-pin uit de MO-reeks op het bord solderen met andere specificaties, maar dezelfde hardware standaard… Dit bevordert uiteraard het hergebruik van hardware.

Het schema van het Nucleo-bord maakt integraal deel uit van de cursus en kan je in bijlage vinden! Bestudeer het schema en maak het je eigen zodat je tijdens het schrijven van firmware steeds weet waar je hardwarematig naar toe wil…

Merk op dat er twee soorten uitbreidingsconnectoren aanwezig zijn op het Nucleo-bord:

1. Arduino Uno compatibele stacking headers
2. ST Morpho connectors

Beide zorgen voor relatief eenvoudige 'plug & play' uitbreidingsmogelijkheden. Eén van de manieren waarop wij zullen werken, is met het Nucleo Extension Shield (zie verder).

r1065698 Tim Kindt

ST-link (en connector)

Op het Nucleo-bord is er een ST-link programmer voorzien. Met die programmer kan je de microcontroller herprogrammeren. Mocht die ST-link niet voorzien zijn, zouden we via een externe (ST-link) programmer onze firmware in de microcontroller moeten programmeren.

Figuur 12: ST-link programmer (bron: st.com)

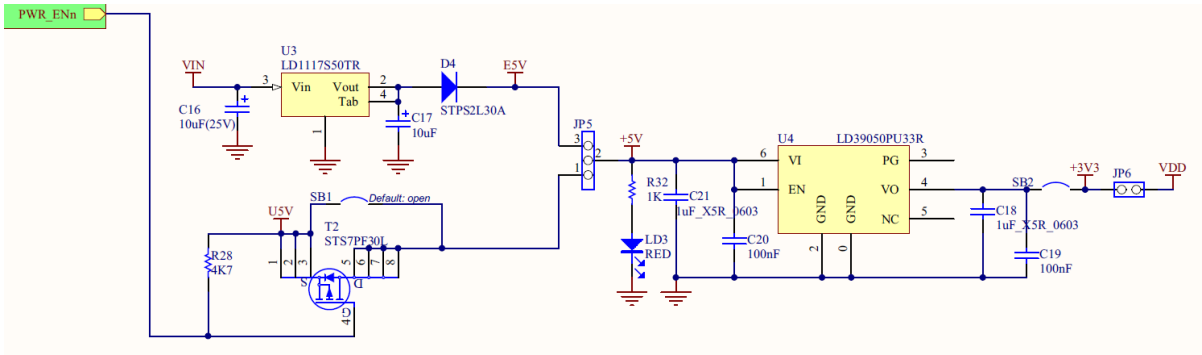
We kunnen ons Nucleo-bord echter ook 'omtoveren' tot een ST-link programmer. Hiervoor moet je de jumpers op CN2 verwijderen en heb je de SWD-connector van CN4 ter beschikking. Op die manier kan je andere borden met een ST microcontroller gaan programmeren.

Hoe opstarten met externe voeding?

Het Nucleo-bord kan je **voeden via de USB-poort die op een computer is aangesloten**. Op die manier kan je totale schakeling 300 mA aan stroom verbruiken. Als je via USB wil voeden zonder dat jouw bord aan een computer hangt (en dus niet geënumereerd wordt), moet je jumper JP1 kortsluiten. Dan mag je schakeling wel slechts 100 mA stroom verbruiken. Meer info vind je in de 'Nucleo user manual' (UM1724 - User manual).

Als je het Nucleo-bord wil **voeden met een voedingsbron die niet uit de USB-poort komt** van een computer, moet je een aantal zaken aanpassen.

Bekijk het gedeelte van het voedingscircuit maar eens:



Figuur 13: voedingscircuit van het Nucleo-bord

Heb je een voedingspanning ter beschikking die nog **niet gestabiliseerd** is op 5Vdc, dan kan je best via VIN binnenkomen met je voedingsbron. Via U3 wordt er dan 5Vdc afgeregeld. Daarna maakt U4 er 3V3 van. Om dit te laten werken moet JP5 pin 2 en 3 verbinden.

Heb je echter wel een **gestabiliseerde 5V** bron ter beschikking, kan je rechtstreeks op de +5V pin binnenkomen. Ook om dit te laten werken moet JP5 pin 2 en 3 verbinden.

Er zijn twee mogelijkheden om een gestabiliseerde 5V bron aan te sluiten:

1. De pinnen van connector CN6 (Arduino):
    a. pin 5 = +5Vdc
    b. pin 6 = GND
2. De pinnen van connector CN7 (Morpho):
    a. pin 24 = +5Vdc
    b. pin 22 = GND

Let wel op! Foute verbindingen op dit niveau kunnen je volledige bord onherroepelijk stuk maken!

Zoek zelf eens uit wat de eigenschappen zijn van U3 en U4. Wat is de bedoeling van PWR_ENn?

r1065698 Tim Kindt

Extension shield voor Nucleo en Arduino

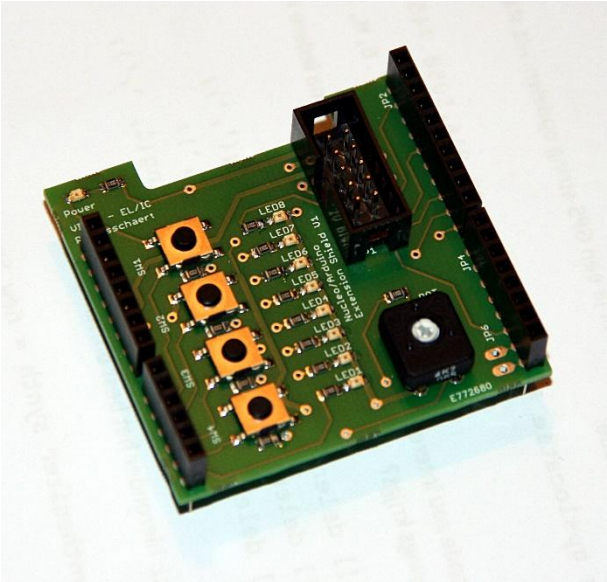
Waarom?

Het Nucleo-bord is ontworpen als prototyping bord. Het voordeel is dat je er nog alle wegen mee uit kan. Het nadeel, dat de weg nog niet gelegd is voor u… :-S

Met andere woorden, je hebt op het naakte Nucleo-bord weinig mogelijkheden om gemakkelijk input te genereren of output zichtbaar te maken. Er is slechts 1 user button en 1 user LED.

Om praktisch te kunnen werken zou het handig zijn dat er 'on board' wat 'user IO' beschikbaar is. Vandaar het Nucleo Extension Shield.

Dat shield is speciaal ontworpen om basisopdrachten i.v.m. microcontrollers vlot te kunnen maken. Het shield bevat 4 drukknoppen, 8 LED's, 1 trimmer en een 10-pin header voor het gebruik van bijvoorbeeld de UART, SPI, I²C,…



Figuur 14: Nucleo Extension Shield

De locatie van de stackable headers is zodanig dat die compatibel is met de Nucleo-borden, maar ook met de Arduino Uno R3 borden.

Merk wel op dat de Nucleo-borden werken op 3,3V. Vandaar dat de pull-up weerstanden van de drukknoppen gekoppeld zijn aan de 3,3V. Het Arduino-bord werkt echter op 5V. Maar omdat de 'minimum input high voltage' van een ATmega328P op 0.6*Vcc (= 3 V) ligt, is 3,3V van de pull-ups voldoende.

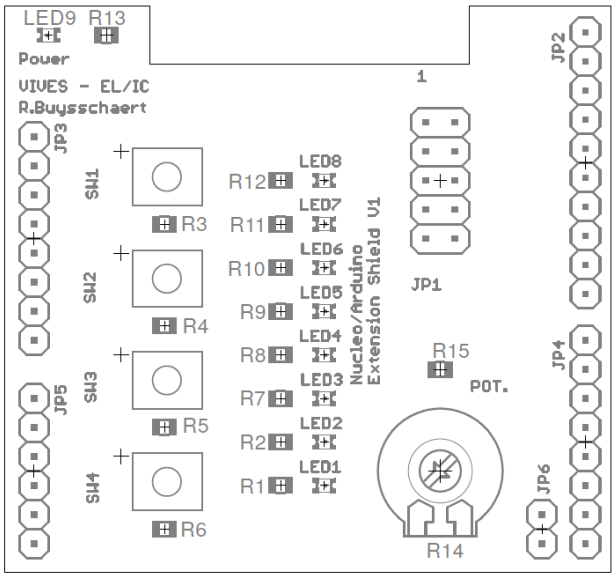
Schema

Het **schema** van het Nucleo Extension Shield **maakt integraal deel uit van de cursus** en is terug te vinden als **bijlage**.

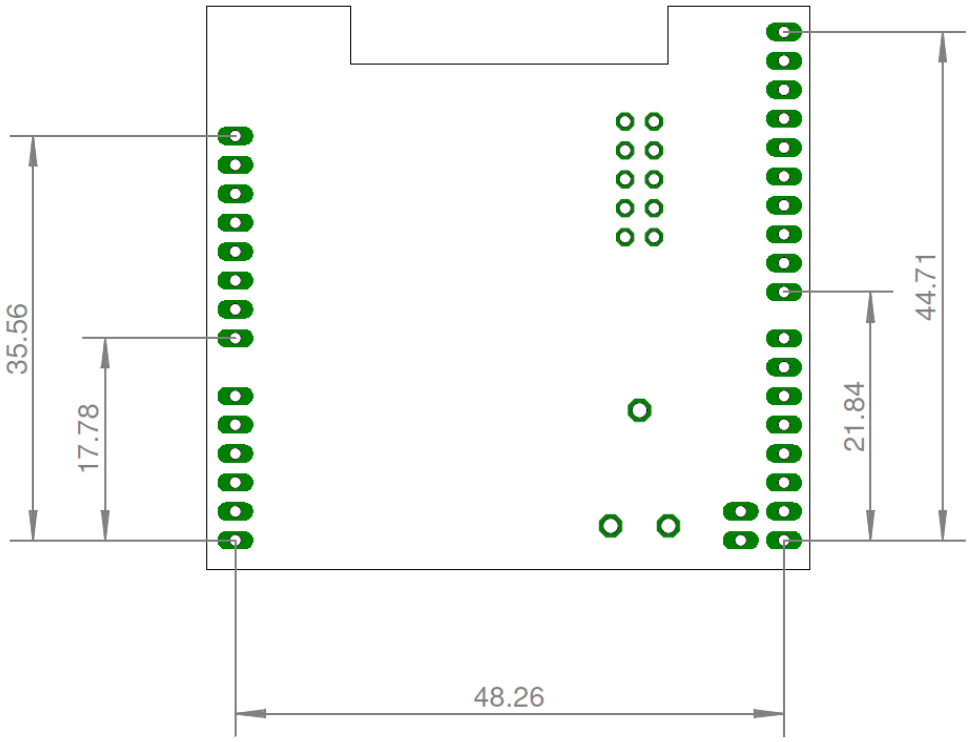
r1065698 Tim Kindt

Board layout

De (gedeeltelijke) board layout kan je hieronder vinden. Je krijgt een Nucleo Extension Shield ter beschikking voor het maken van de labo-oefeningen. **Let wel op bij het verwijderen van het shield. Zorg dat de stackable header pinnen niet geplooid geraken bij het loskomen van de shield. Werk voorzichtig! Potje breek = potje betaal…**



Figuur 15: Nucleo Extension Shield - top view



Figuur 16:Nucleo Extension Shield – measurements

r1065698 Tim Kindt

Keil IDE

Wat is Keil?

Zoals de fabrikant zelf zegt: “The Keil ARM Microcontroller Development Kit (MDK-ARM®) tools allow to create applications in C and C++ for ARM- and Cortex-M processor-based devices.”

Het is dus een **development kit** waarin zowel voorbeeldcode, een compiler als een IDE (Integrated Development Environment) aanwezig zijn.

Je kan de Keil MDK met **µVision** downloaden van hun website:
https://www.keil.com/demo/eval/arm.htm

Er is een gratis versie van de MDK beschikbaar. Die heeft wel enkele beperkingen, maar dit vormt voorlopig geen belemmering.

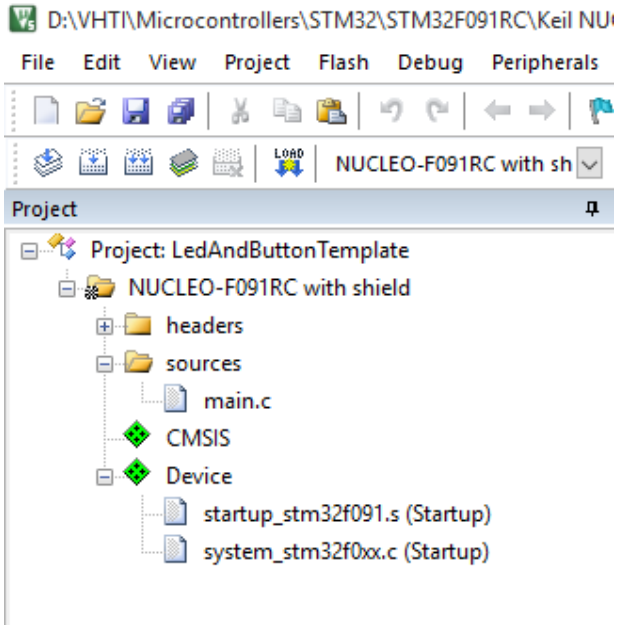
Om met het Nucleo-bord te werken, ben je echter niet verplicht om met de Keil software te werken. Het is wel zo dat de andere softwarepakketten niet in deze cursus behandeld zullen worden.

Toepassingsvoorbeeld: mijn eerste project

Om vlot van start te kunnen gaan, start je best met een voorbeeldproject (projectsjabloon). Dat sjabloon wordt ter beschikking gesteld via Toledo en kan dienen als opstart voor al je komende projecten.

Het is echter wel zo dat er verschillende manieren zijn om je project op te bouwen in Keil. Het sjabloon bevat er één van. Naarmate de cursus vordert kan het zijn dat er ook nog andere projectopstellingen gebruikt zullen worden.

Je kan het projectsjabloon openen door te dubbelklikken op het ‘.uvprojx’-bestand.



Figuur 17: screenshot projectopbouw in Keil µVision

r1065698 Tim Kindt

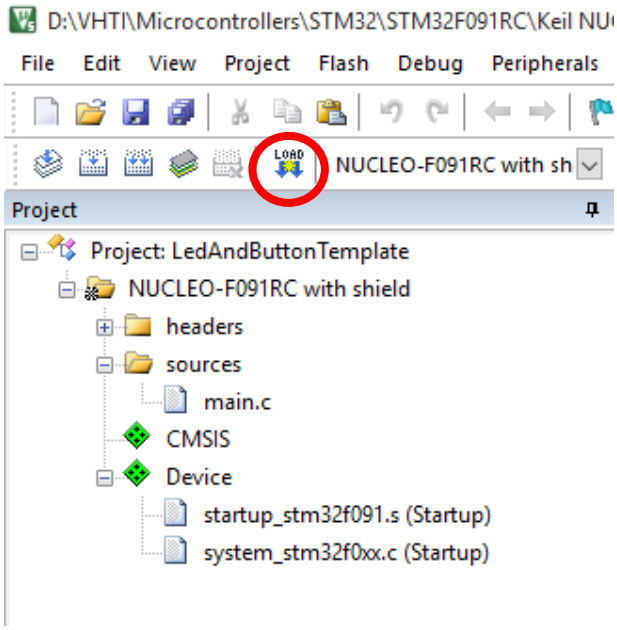
Dubbelklik op de ‘main.c’ zodat je de C-code van het programma kan bekijken. Probeer te begrijpen wat er staat…

```c
int main(void)
{
  SystemClock_Config();

  InitIo();

  while (1)
  {
    // SW1 actief?
    if((GPIOA->IDR & GPIO_IDR_1) != GPIO_IDR_1)
        GPIOC->ODR = GPIOC->ODR | GPIO_ODR_0;
    else
        GPIOC->ODR = GPIOC->ODR & ~GPIO_ODR_0;
  }
}
```

Figuur 18: Screenshot main.c

r1065698 Tim Kindt

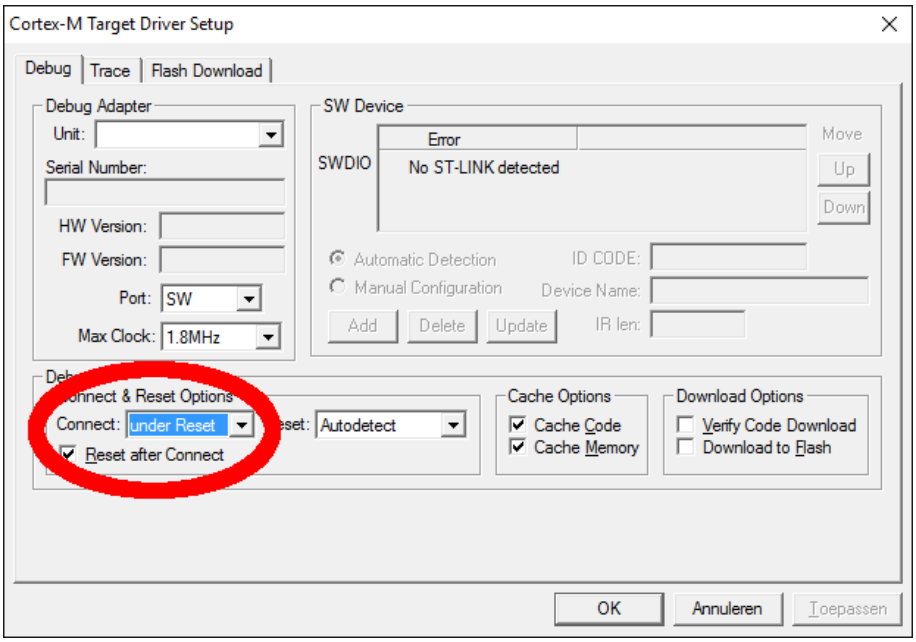
Probeer de firmware te uploaden naar uw Nucleo-bord met Extension Shield. Je kan dat via de knop “load” in µVision.



Figuur 19: aanduiding 'load-knop' in µVision

Test de code uit op het bordje…

Merk op! **Als je problemen ervaart bij het programmeren**, bekijk dan eerst of de drivers correct zijn geïnstalleerd. Als dat het geval is en je ervaart nog 'target-problemen', bekijk dan eens de instellingen van de ST-link debugger:



r1065698 Tim Kindt

Eenvoudige output genereren

Een pin als output

Basisinstellingen

Na het **resetten** van de microcontroller zijn alle pinnen **ingesteld als input**. Op die manier kunnen ze minst schade aanrichten op hardware vlak. Een input gedraagt zich immers als 'iets hoogimpedant' en zal dus geen grote belasting vormen voor eventueel gekoppelde hardware.

Om en pin van de microcontroller **als output te plaatsen** moeten we enkele instellingen doen.

Stel dat we LED1 van het Nucleo Extension Shield op output willen plaasten, dan kan dat met volgende code:

```
// Clock voor GPIOC inschakelen.
RCC->AHBENR = RCC->AHBENR | RCC_AHBENR_GPIOCEN;

// LED1 op output zetten
GPIOC->MODER = (GPIOC->MODER & ~GPIO_MODER_MODER0) | GPIO_MODER_MODER0_0;
```

Figuur 20: code snippet voor het instellen van LED1

Zoek uit wat de registers betekenen en welke bewerkingen gedaan worden!

Maximum stroom

Elke IO mag maximum 25mA leveren (source) of opnemen (sink). De som van alle stromen in de microcontroller mag niet hoger zijn dan 80mA. Die info kan je terugvinden in de datasheet van de ST32F091RC.

Schakelsnelheid

Om het energieverbruik van de microcontroller zoveel mogelijk te beperken kan je een stijg- en daaltijden van de outputs instellen. Hoe sneller een output moet schakelen, hoe meer energie daarvoor nodig is.

De schakelsnelheid stel je in via het OSPEEDRx register. **Zoek uit** hoe dit ingesteld staat na reset en bekijk de maximum schakelsnelheden als we een voedingspanning van 3,3V gebruiken…

Output een waarde geven

Vul hieronder aan hoe je de LED1 zou kunnen inschakelen en uitschakelen:

```
GPIOC->ODR =
```

```
GPIOC->ODR =
```

r1065698 Tim Kindt

Maak een knipper-LED

Als kort maar krachtig bewijs dat de klokinstellingen en de outputs van de microcontroller correct zijn ingesteld, kan je een knipper-LED maken.

Eén mogelijke manier is:

```
// Oneindig lang lopen...
while (1)
{
  // LED1 toggle'n
  GPIOC->ODR = GPIOC->ODR ^ GPIO_ODR_0;

  // een tijdje wachten
  for(ticks = 0; ticks < 2000000; ticks++)
    __NOP();
}
```

Figuur 21: Code snippet knipper-LED

Wat betekent `__NOP()`? Van welk type zou je `ticks` kiezen?

r1065698 Tim Kindt

To do

Wat kan je **zelf nog doen**? Om gemakkelijk en flexibel 'real world problems' aan te pakken, kan het handig zijn dat je volgende functies programmeert (en in een bibliotheek opslaat):

- `void WaitForMs(uint32_t time);`
- `void ByteToLeds(uint8_t data);`

Die functies doen exact wat ze zeggen: een gegeven aantal milliseconden wachten en 8-bit data op de 8 LED's van het Nucleo Extension Shield zichtbaar maken.

Probeer die functies eens uit in een 8-bit teller… Je zal deze functies dikwijls kunnen gebruiken.

Eenvoudige input verwerken

Een pin als input

Basisinstellingen

Na **reset** zijn de pinnen van de microcontroller automatisch op **input ingesteld**. We kunnen die dus onmiddellijk gebruiken. Het is wel zo dat je de 'poort' waarvan een bepaalde pin deel uitmaakt, nog van **klok moet voorzien**. Anders is die poort uitgeschakeld.

De klok voorzien voor SW1 van het Nucleo Extension Shield kan zo:

```
// SW1 is standaard input na power on reset, wel nog klok voorzien.
RCC->AHBENR = RCC->AHBENR | RCC_AHBENR_GPIOAEN;
```

Figuur 22: Code snippet voor het voorzien van een klok op poort A

Pull-up/Pull-down

Om de ontwerper gemakkelijk inputs te laten verwerken, voorziet de microcontroller de mogelijkheid om interne pull-up of pull-down weerstanden te gebruiken. De instelling gebeurt via PUPDR. **Zoek uit** wat de instelling is na reset in de datasheet.

Het Nucleo Extension Shield heeft externe pull-up weerstanden on board. De interne weerstanden van de STM32F091RC zullen we dus in dit geval niet gebruiken.

r1065698 Tim Kindt

Toestand van een knop detecteren

Als basisvoorbeeld van het verwerken van input, kan de toestand van een knop (SW1) ingelezen worden en op een LED (LED1) weergegeven worden. Eén mogelijke manier vind je hieronder:

```
if((GPIOA->IDR & GPIO_IDR_1) != GPIO_IDR_1)
    GPIOC->ODR = GPIOC->ODR | GPIO_ODR_0;
else
    GPIOC->ODR = GPIOC->ODR & ~GPIO_ODR_0;
```

Figuur 23: Code snippet inlezen knop

Zoek zelf uit wat her precies gebeurt…

To do

Wat kan je **zelf nog doen** om alles onder de knie te krijgen?

- Maak een 8-bit teller die telt indien knop wordt ingedrukt
- Detecteer een 'rising edge' van een knop via een hulpvariabele

Basisklokinstelling van de STM32F091RC

Klok

De microcontroller kan enkel zijn ‘werk’ doen (instructies uitvoeren) als die gevoed wordt met een klok. De klok zorgt ervoor dat de microcontroller een instructie aanvat en afhandelt, daarna een volgende instructie, enzovoort.

Hoe sneller de klokpulsen komen, hoe sneller de microcontroller werkt. Er is natuurlijk een bovengrens aan die snelheid. Die **bovengrens** wordt bepaald door onder andere:

- de **parasitaire** capaciteiten op allerhande baantjes
- de **imperfectie** van de µC ‘die’
- het maximum gewenste **energieverbruik**
- …

Het kan dus perfect zijn dat je een microcontroller gebruikt op een klokfrequentie die lager is dan zijn maximum werkbare klokfrequentie. Het is aan de ontwerper van het systeem, of software, om dat te bepalen.

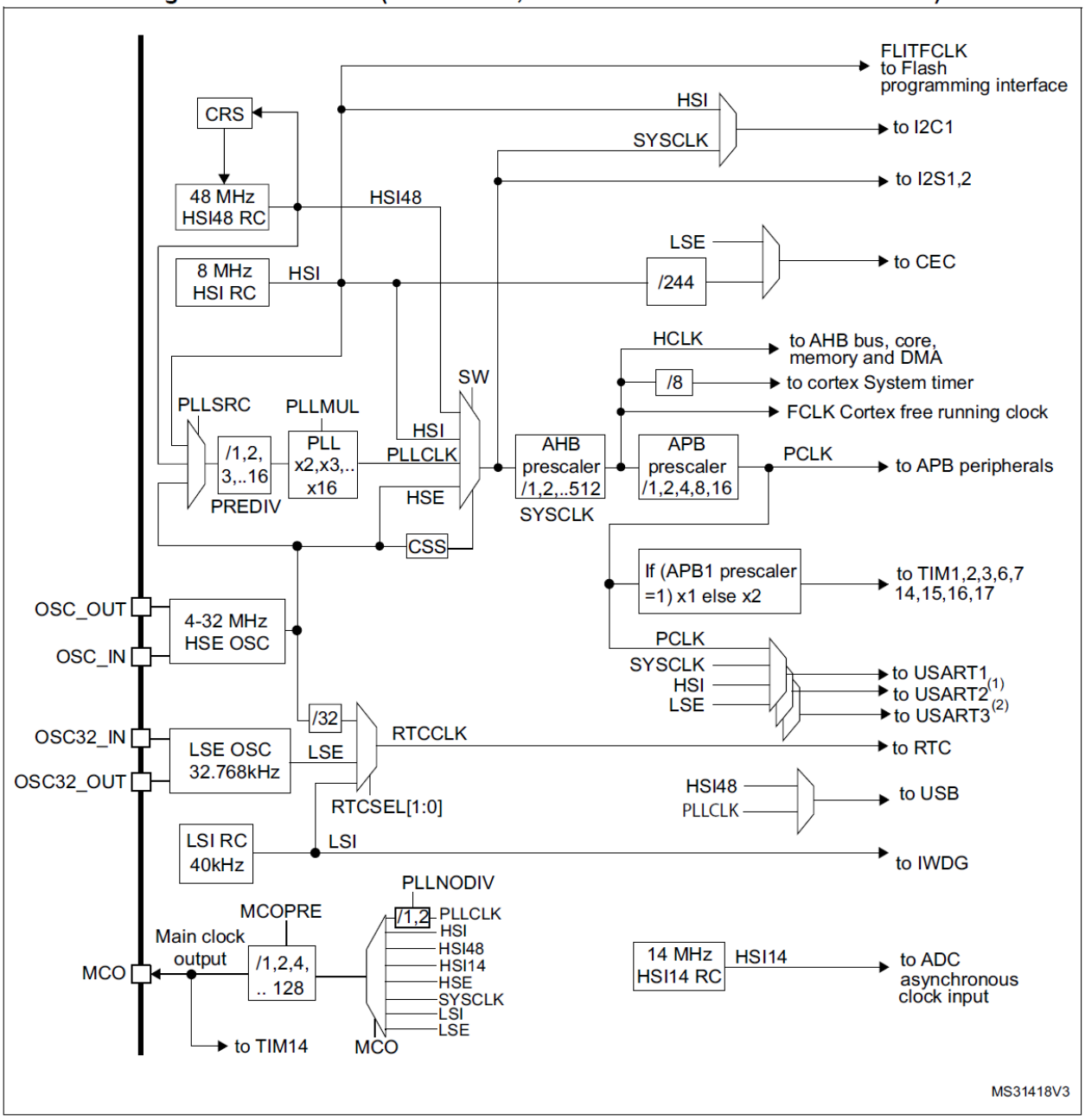
r1065698 Tim Kindt

Clock tree

Omdat het instellen van de klok soms vrij ingewikkeld kan zijn, voorzien de meeste fabrikanten een visuele voorstelling van het hele kloksysteem. De zogenaamde '**clock tree**'. Onderstaande clock tree is heel belangrijk! Zonder een klok, of de juiste klok, geen microcontrollers…

Zoek uit langs welk pad we de CPU-kern met **48MHz** zouden kunnen voeden. Let op, gebruik **niet de HSI48 RC** klok. Die is niet stabiel genoeg en wensen we niet te gebruiken.

Figure 12. Clock tree (STM32F04x, STM32F07x and STM32F09x devices)



1. Not available on STM32F04x devices.
2. Not available on STM32F04x and STM32F07x devices

FCLK acts as Cortex®-M0's free-running clock. For more details refer to the *ARM Cortex™-M0 r0p0 technical reference manual (TRM).*

Figuur 24: Clock tree STM32F091RC (bron: RM0091)

r1065698 Tim Kindt

Instellingen om de µC van een klok te voorzien

Op het moment dat de microcontroller gereset wordt, wordt gebruik gemaakt van een basisklokinstelling. Onze gebruikte microcontroller heeft een on board high speed RC oscillator van 8MHz, waardoor een extern kristal niet echt hoeft gebruikt te worden (ten zij je een heel stabiele klok wenst).

Die interne high speed RC klok werkt dus op 8MHz. Die 8MHz kan via **PLL** vermenigvuldigers en **prescalers** omgezet worden naar 48MHz. De hoogste snelheid waarop de STM32F091RC kan werken.

Na het opstarten van de microcontroller moet de code voorzien worden om de 48MHz klok te gaan instellen.

Dit kan zoals hieronder aangegeven:

```
58  void SystemClock_Config(void)
59  {
60    RCC->CR |= RCC_CR_HSITRIM_4;
61    RCC->CR  |= RCC_CR_HSION;
62    while((RCC->CR & RCC_CR_HSIRDY) == 0);
63
64    RCC->CFGR &= ~RCC_CFGR_SW;
65    while((RCC->CFGR & RCC_CFGR_SWS) != RCC_CFGR_SWS_HSI);
66
67    RCC->CR &= ~RCC_CR_PLLON;
68    while((RCC->CR & RCC_CR_PLLRDY) != 0);
69
70    RCC->CFGR |= RCC_CFGR_PLLSRC_HSI_PREDIV;
71    RCC->CFGR2 |= RCC_CFGR2_PREDIV_DIV2;
72    RCC->CFGR |= RCC_CFGR_PLLMUL12;
73
74    FLASH->ACR |= FLASH_ACR_LATENCY;
75
76    RCC->CR |= RCC_CR_PLLON;
77    while((RCC->CR & RCC_CR_PLLRDY) == 0);
78
79    RCC->CFGR |= RCC_CFGR_SW_PLL;
80    while((RCC->CFGR & RCC_CFGR_SWS) != RCC_CFGR_SWS_PLL);
81
82    RCC->CFGR |= RCC_CFGR_HPRE_DIV1;
83    RCC->CFGR |= RCC_CFGR_PPRE_DIV1;
84
85    SystemCoreClockUpdate();
86    SysTick_Config(48000);
87
88  }
```

Figuur 25: code snippet van de klokinstellingen

Bekijk deze code goed en **zoek uit** wat alle registers betekenen, hoe de bitbewerkingen effect hebben, wat betekenen '->',…?

Ga hier niet licht over! Dergelijke code zal steeds opnieuw voorkomen als we de ARM microcontrollers gaan programmeren. **Dit niet begrijpen is geen optie! ;-)**

r1065698 Tim Kindt

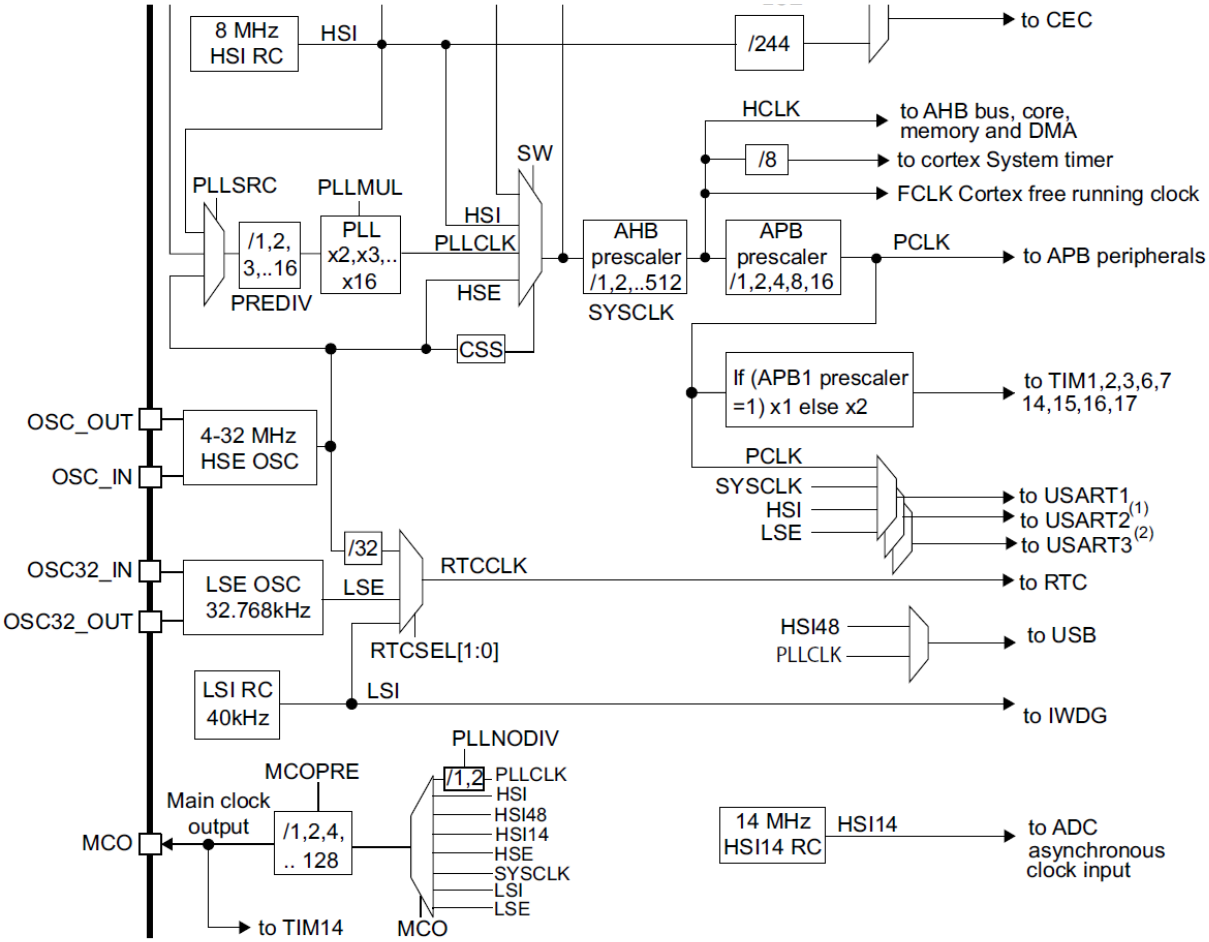
Toepassingsvoorbeeld: watchdog timer

De meeste microcontrollers worden ingezet bij toepassingen die weinig of zelfs geen user interface hebben. Daarenboven staan ze dicht bij de hardware en zijn ze essentieel in een groter systeem. Om 'vastgelopen' software van een microcontroller automatisch te **resetten** kan je gebruik maken van een **watchdog timer**.

Een watchdog is een soort '**waakhond'**. Als de timer verbonden met de watchdog, niet tijdig gereset wordt, gaat die timer zelf het initiatief nemen om heel de microcontroller te resetten in de hoop dat de herstart het systeem weer 'op de rails' krijgt.

Ook de STM32F091RC heeft zo'n watchdog timer.

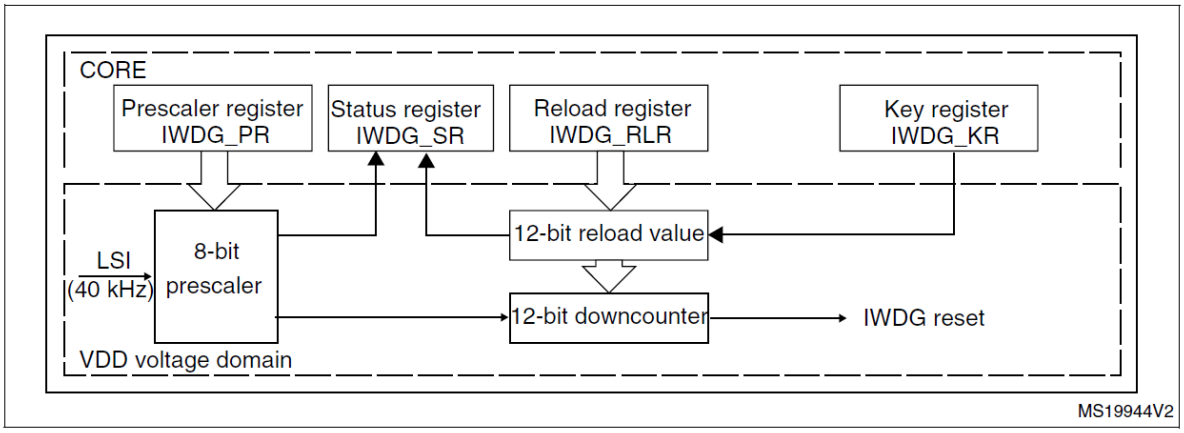
De Independant WatchDoG (IWDG) wordt gevoed met een aparte oscillator op 40kHz. Kan je de juiste locatie aanduiden op de clock tree?



Het **voordeel** van de 'dedicated clock' is dat een foute klokinstelling voor de core, geen invloed heeft op de watchdog timer.

Het blokschema van de watchdog ziet je hieronder:

r1065698 Tim Kindt



The watchdog function is implemented in the CORE voltage domain that is still functional in Stop and Standby modes.

Figuur 26: Independant watchdog blokschema (bon: RM0091)

De watchdog kan gestart/enabled worden door 0x0000CCCC te schrijven naar het Key register. De waarde van het Reload register wordt in de 12-bit downcounter geladen als er 0x0000AAAA naar het Key register geschreven wordt. Die info is meegegeven door de fabrikant. Zoek maar eens uit hoe alles zit in de datasheet van de microcontroller…

Om op zijn beurt toegang te krijgen tot het Reload register moet je eerst 0x00005555 schrijven naar het Key register…

Op het moment dat de 12-bit downcounter nul bereikt, wordt de microcontroller gereset.

Als je dan een watchdog reset wil na iets meer dan 6 seconden, kan je volgende code gebruiken:

```
void InitWatchdog(void)
{
  IWDG->KR = 0x0000CCCC;
  IWDG->KR = 0x00005555;
  IWDG->PR = IWDG_PR_PR;
  IWDG->RLR = 1000;
  while(IWDG->SR != 0);
  IWDG->KR = 0x0000AAAA;
}

void ResetWatchdog(void)
{
  IWDG->KR = 0x0000AAAA;
}
```

Figuur 27: Code snippet watchdog timer

Zoek zelf uit wat al die code **exact** doet… Gebruik daarvoor de info uit de datasheet. Kan je dit voorbeeld verwerken in een echt werkend programma (in combinatie met SW1)?

r1065698 Tim Kindt

Eenvoudige interrupts

Wat is een interrupt?

Een interrupt is een verzoek aan de processor om onmiddellijk zijn hoofdprogramma te verlaten en een interrupt routine af te handelen. Die 'interrupt handler' verwerkt de situatie, waarna de processor opnieuw met zijn hoofdprogramma kan verder gaan.

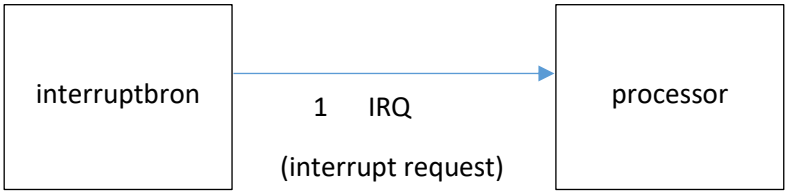
Vlak vóór het verlaten van het hoofdprogramma worden de huidige **program counter** en een aantal andere essentiële registers, **weggeschreven naar de stack**. Daarna wordt de program counter ingesteld op het adres van de interrupt (vector table) om zo **naar de interrupt handler te verwijzen**.

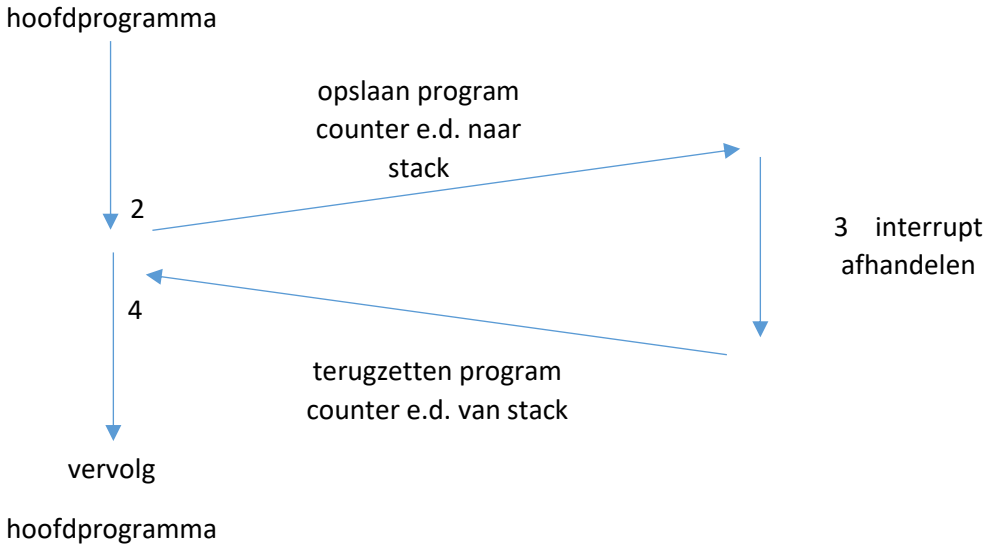
Na het afhandelen van de interrupt worden de gegevens van de stack teruggezet zoals voorheen en **kan het hoofdprogramma verdergaan** zoals dit het geval was vóór de interrupt.

Er zijn verschillende bronnen van interrupts. Ze kunnen zowel softwarematig als hardwarematig zijn. Vooral 'peripherals' (randapparatuur zoals de seriële poort), timers en andere input kunnen interrupts veroorzaken.

Het voordeel van het werken met interrupts is dat de aanvrager van de interrupt heel snel op zijn wenken bediend wordt!

Het **interruptconcept** kan je als volgt voorstellen:





r1065698 Tim Kindt

NVIC (Nested Vectored Interrupt Controller)

De NVIC is de schakel tussen de interruptbron en de CPU. Hij (of zij ;-)) kan 32 interruptkanalen in goede banen leiden. Daarbovenop zijn er nog 16 Cortex M0 – interruptlijnen.

Die 32 interruptkanalen kunnen volgens **4 programmeerbare prioriteitenniveau's** (0 tot 3) afgehandeld worden. Als er zich op het zelfde moment twee interrupts voordoen die dezelfde programmeerbare prioriteit hebben meegekregen, wordt eerst de interrupt afgehandeld van diegene met laagste niet programmeerbare prioriteit in de vector table (zie datasheet p218).

Priority level voorbeeld

Als er **op hetzelfde moment een interrupt** binnenkomt van EXTI1 (niet programmeerbare prioriteit 12 en adres 0x0000 0054) en SPI1 (niet programmeerbare prioriteit 32 en adres 0x0000 00A4), waarvan de **programmable interrupt priority beide op 0 stonden**, dan wordt eerst de EXTI1 interrupt afgehandeld en daarna de SPI1 interrupt.

In code wordt dit:

```
NVIC_SetPriority(EXTI0_1_IRQn,0);

NVIC_SetPriority(SPI1_IRQn,0);
```

Figuur 28: programmable interrupt priority levels gelijk zetten

Als je om programmatorische redenen **toch eerst SPI1** wil afhandelen, kan je via de programmable interrupt priority de SPI1 een **lagere priority instellen**. Zoals:

```
NVIC_SetPriority(EXTI0_1_IRQn,1);

NVIC_SetPriority(SPI1_IRQn,0);
```

Figuur 29: programmable interrupt priority levels verschillend maken

Startup

Bij de opstart van de microcontroller moet er gereset worden en naar de main-functie gegaan worden. Om dat correct te laten verlopen moet de Vector Mapping goed zijn. Ook voor de interrupts wordt die mapping belangrijk. Bekijk maar eens het bestand 'startup_stm32f091xc.s'…

OPM:

"The DCD directive allocates one or more words of memory, aligned on four-byte boundaries…"

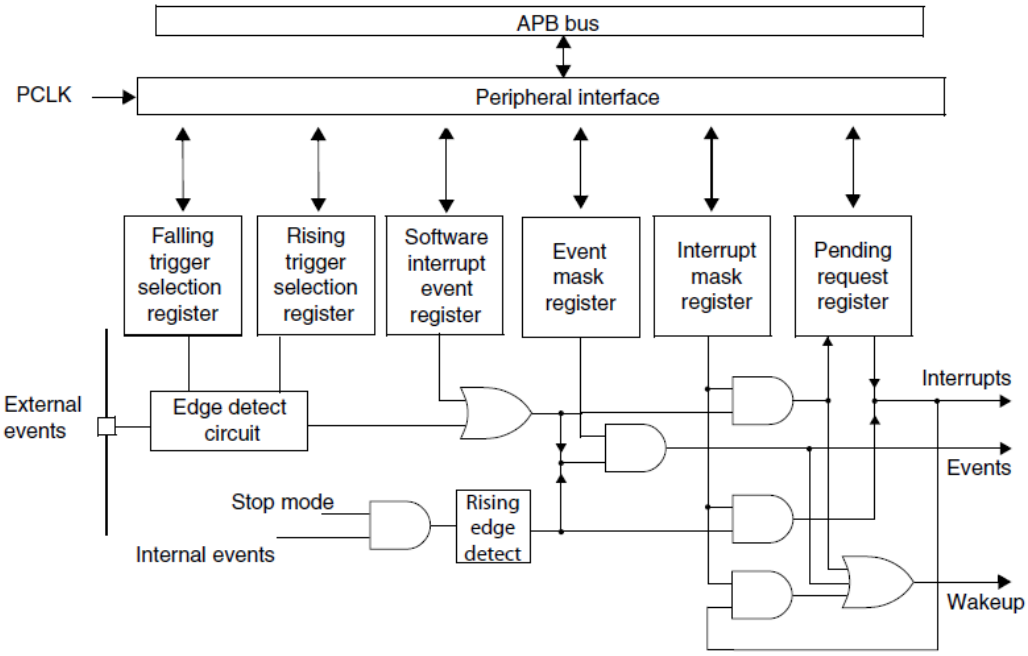
```
__Vectors       DCD     __initial_sp                ; Top of Stack
                DCD     Reset_Handler               ; Reset Handler
                DCD     NMI_Handler                 ; NMI Handler
                DCD     HardFault_Handler           ; Hard Fault Handler
                DCD     0                           ; Reserved
```

r1065698 Tim Kindt

Basis instellingen voor een External interrupt

Eén type interrupts die de processor kan verwerken, komt van de EXTernal Interrupt and event controller. Via deze weg kan je gemakkelijk interrupt van 32 bronnen (23 externe en 9 interne) tot bij de processor loodsen.

Het principeschema ziet er als volgt uit. Kan je de verschillende blokjes bespreken? Wat is bijvoorbeeld de 'weg' van een stijgende flank op een pin tot bij de core voor een externe interrupt?



Figuur 30: external interrup and event controller blokschema

Andere interrupt bronnen

Andere interrupt bronnen kan je opzoeken in de datasheet. Diegene die zeker nog zullen aanbod komen in de cursus zijn:

- Watchdog timer interrupt
- SPI interrupt
- I²C interrupt
- UART interrupt
- Timer interrupts
- …

Telkens wanneer je een interrupt zal willen gebruiken, moeten alle betrokken registers correct ingesteld worden. Hiervoor kan je best de datasheet dichtbij de hand nemen…

r1065698 Tim Kindt

External interrupts en knoppen

We kunnen de external interrupts gebruiken om via een knop een interrupt te genereren. Hieronder de code voor het lezen van **SW2** op het Nucleo Extension Shield, via interrupts:

```
45    // SYSCFG clock enable.
46    RCC->APB2ENR |= RCC_APB2ENR_SYSCFGEN;
47
48    // PA4 (SW2) koppelen aan EXTI 4.
49    SYSCFG->EXTICR[1] |= SYSCFG_EXTICR2_EXTI4_PA;
50
51    // Falling edge detecteren.
52    EXTI->FTSR = EXTI->FTSR | EXTI_FTSR_TR4;
53
54    // Interrupt toelaten.
55    EXTI->IMR = EXTI->IMR | EXTI_IMR_MR4;
56
57    // Eén van de 4 prioriteiten kiezen. Als er twee
58    // interrupts zijn met dezelfde prioriteit, wordt
59    // eerst diegene afgehandeld die de laagste
60    // 'position' heeft (zie vector table NVIC in datasheet).
61    NVIC_SetPriority(EXTI4_15_IRQn,0);
62
63    // Interrupt effectief toelaten.
64    NVIC_EnableIRQ(EXTI4_15_IRQn);
```

En de interrupt handler:

```
62  // Interrupt handler aanmaken. De definities van
63  // de handler kan je vinden in startup_stp32f091xc.s.
64  void EXTI4_15_IRQHandler(void)
65  {
66    // Als het een interrupt is van PA4, de LED's inschakelen.
67    if((EXTI->PR & EXTI_PR_PR4) == EXTI_PR_PR4)
68    {
69      // Interrupt (pending) vlag wissen door
70      // een 1 te schrijven...
71      EXTI->PR |= EXTI_PR_PR4;
72
73      // LED's inschakelen als bewijs van interrupt.
74      GPIOC->ODR = GPIOC->ODR | GPIO_ODR_0;
75      GPIOB->ODR = GPIOB->ODR | GPIO_ODR_3;
76      GPIOB->ODR = GPIOB->ODR | GPIO_ODR_5;
77      GPIOB->ODR = GPIOB->ODR | GPIO_ODR_4;
78      GPIOB->ODR = GPIOB->ODR | GPIO_ODR_10;
79      GPIOA->ODR = GPIOA->ODR | GPIO_ODR_8;
80      GPIOC->ODR = GPIOC->ODR | GPIO_ODR_7;
81      GPIOB->ODR = GPIOB->ODR | GPIO_ODR_6;
82    }
83  }
```

Kan je alle info van de gebruikte registers bovenaan **opzoeken in de datasheets en verklaren** wat de C-code precies doet? Dergelijke redeneringen zijn essentieel!

To do

- Maak een start/stop met een knop en interrupt-code

r1065698 Tim Kindt

Registers bundelen in structs

Wat is een struct?

Een struct is een 'complex' data type. Het definieert variabelen die samen één geheel vormen. De verschillende onderdelen uit een struct kunnen via één overkoepelende naam bereikt worden.

Het voordeel van een struct is vooral dat de programmeur zijn/haar data op een meer logische manier kan ***struct***ureren…. ;-)

Een voorbeeldje:

```
#if !defined(PERSOON_DEFINED)
  #define PERSOON_DEFINED

  typedef struct
  {
    char voornaam[50];
    char achternaam[50];
    float gewicht;
  }Persoon;
#endif
```

Bovenstaande code kan je eventueel via een header-file toevoegen aan je project. Op die manier blijft alles overzichtelijk en gestructureerd.

Het nieuw aangemaakt type *Persoon*, kan nu gebruikt worden in je hoofdprogramma. Bijvoorbeeld '*mezelf*' aanmaken:

```
19   Persoon mezelf;                    // nieuw Persoon aanmaken
```

Daarna gebruiken:

```
28   //Struct invullen met de gewenste data.
29   sprintf(mezelf.voornaam, "Ruben");
30   sprintf(mezelf.achternaam, "Buysschaert");
31   mezelf.gewicht = 78.5685;
```

sprintf is een functie (uit de stdio.h) waarmee je een data kan kopiëren naar een bepaald character array. Het is ook mogelijk om variabelen daarin te verwerken. Bijvoorbeeld:

```
78 void PersoonToUsart(Persoon * pPersoon)
79 {
80   char tekst[21];    // plaats voorzien voor 20 karakters
81
82   StringToUsart(pPersoon->voornaam);
83   StringToUsart(" ");
84   StringToUsart(pPersoon->achternaam);
85   sprintf(tekst, ": %.1f kg.", pPersoon->gewicht);
86   StringToUsart(tekst);
87   CrLfToUsart();
88 }
```

r1065698 Tim Kindt

Pointers in C

Ook in C-taal worden variabelen opgeslagen op een bepaald adres in de RAM.

Veronderstel bijvoorbeeld een variabele `getal`: 8-bit breed, opgeslagen op adres 12543 en als waarde 138. Dit kan je in de RAM min of meer grafisch voorstellen door:

`getal` (adres = 12543)

138

Het adres van een variabele in C-taal kan je opvragen via het **&**-teken (ampersand). Zo kan je stellen dat:

`&getal` gelijk moet zijn aan 12543.

In C-taal bestaan er speciale types **die als inhoud een adres bevatten** naar een ander type. Zoiets heet met een **pointervariabele**. In C# kan je dit vergelijken met een reference type.

Om een pointervariabele te maken gebruik je het *****-teken (asterisk). Als je een pointer naar een unsigned 8-bit getal wil maken kan je dit als volgt doen:

```
uint8_t* getalPointer;
```

Om daarin dan het adres van een dergelijke 8-bit variabele te plaatsen kan je:

```
getalPointer = &getal;
```

Als je dit een grafische voorstelling in de RAM wil geven, kan je het volgende doen:

`getalPointer` (adres = 14592)

12543

Daarna kan je *`getalPointer`* gebruiken op twee manieren:

1. Je kan **de inhoud van *`getalPointer`*** opvragen. Die inhoud is dus **het adres van *`getal`* (12543)**. Dit doe je zo:

   ```
   … = getalPointer;
   ```

2. Je kan de **inhoud waarnaar *`getalPointer`* wijst** opvragen. Die inhoud is de **inhoud van *`getal (138)`***.

   ```
   … = *getalPointer;
   ```

   Het bovenstaande is wel verwarrend. Waarom? **Omdat het *-teken dus twee betekenissen heeft!** Je kan het gebruiken bij de aanmaak van een variabele. Dan creëer je dus een pointervariabele. Maar je kan het ook gebruiken bij een bestaande pointervariabele. Dan krijg je de data waarnaar de pointervariabele verwijst… **Probeer je dit goed voor te stellen! Maak een tekening van hoe de variabelen zich in de RAM gedragen!** Enkel dan kan je pointers echt goed begrijpen.

r1065698 Tim Kindt

Hieronder een codevoorbeeld met pointers.

```
20 uint32_t numberOfRecords = 354;
21 uint32_t* pointerToNumberOfRecords;
```

```
40     // Pointer spelletjes...
41     sprintf(tekst, "Adres van numberOfRecords: %d.", (uint32_t)(&numberOfRecords));
42     StringToUsart(tekst);
43     CrLfToUsart();
44
45     sprintf(tekst, "Inhoud van numberOfRecords: %d.", numberOfRecords);
46     StringToUsart(tekst);
47     CrLfToUsart();
48
49     sprintf(tekst, "Adres van pointerToNumberOfRecords: %d.", (uint32_t)(&pointerToNumberOfRecords));
50     StringToUsart(tekst);
51     CrLfToUsart();
52
53     sprintf(tekst, "Inhoud van pointerToNumberOfRecords: %d.", (uint32_t)pointerToNumberOfRecords);
54     StringToUsart(tekst);
55     CrLfToUsart();
56
57     sprintf(tekst, "Data naar waar pointerToNumberOfRecords wijst: %d.", *pointerToNumberOfRecords);
58     StringToUsart(tekst);
59     CrLfToUsart();
60     CrLfToUsart();
```

Het *-teken kan dus inderdaad twee betekenissen hebben. Ofwel wordt het gebruikt om een pointer naar een bepaald type aan te maken (lijn 21), ofwel voor het opvragen van de data die aan een bepaalde pointervariabele is gekoppeld (lijn 57).

Het &-teken wordt gebruikt om van een bepaalde variabele het adres op te vragen. Als we bovenstaande code uitvoeren krijg je dit te zien op je computer:

COM18 - PuTTY

```
Adres van numberOfRecords: 536870920.
Inhoud van numberOfRecords: 354.
Adres van pointerToNumberOfRecords: 536870924.
Inhoud van pointerToNumberOfRecords: 536870920.
Data naar waar pointerToNumberOfRecords wijst: 354.
```

r1065698 Tim Kindt

Hoe gebruikt CMSIS structs en pointers om alles overzichtelijker te maken?

De CMSIS (Cortex Microcontroller Software Interface Standard) maakt gebruik van structs en pointers om het programmeren gemakkelijker te maken.

Om de samenwerking tussen stucts en pointer beter te begrijpen, kan je onderstaand voorbeeld bekijken. We maken een 'domoticasysteem' voor een huis met 2 verdiepingen. Op die twee verdiepingen zijn telkens 4 lampen voorzien.

```
16 typedef struct{
17   // Hier stellen de 4 LSB's de 4 lampen voor op het gelijkvloers.
18   // De 4 MSB's zijn 'reserved' en moeten steeds op nul staan.
19   uint8_t verlichtingGelijkvloers;
20
21   // Hier stellen de 4 LSB's de 4 lampen voor op de eerste verdieping.
22   // De 4 MSB's zijn 'reserved' en moeten steeds op nul staan.
23   uint8_t verlichtingEersteVerdieping;
24 }DomoticaSysteem;
25
26
27 DomoticaSysteem huisVanPapaEnMama;    //  ;-)
28 DomoticaSysteem* huisVanDeKindjes;    //  :-p
```

```
39   // Schakel alle vier de lichten op het gelijkvloers aan.
40   huisVanPapaEnMama.verlichtingGelijkvloers = 0x0F;
41   // Schakel alle lichten boven uit.
42   huisVanPapaEnMama.verlichtingEersteVerdieping = 0x00;
43
44   // Omdat de kindjes nog thuis wonen, kan je bijvoorbeeld via
45   // een pointer naar 'huisVanPapaEnMama' ook diezeflde lichten
46   // schakelen... Let op: het gaat dus wel degelijk over dezelfde
47   // lichtpunten! Je schakelt ze gewoon via een andere variabele.
48   huisVanDeKindjes = &huisVanPapaEnMama;
49   // Schakel alle lichten op het gelijkvloers uit.
50   // Merk op dat je nu geen '.' meer mag gebruiken om binnen te
51   // treden in de struct, maar een '->'!!! Verder blijft de
52   // redenering hetzelfde.
53   huisVanDeKindjes->verlichtingGelijkvloers = 0x00;
54   // Schakel 1 licht aan op de eerste verdieping...
55   huisVanDeKindjes->verlichtingEersteVerdieping |= 0x01;
```

r1065698 Tim Kindt

Als voorgaande wat duidelijk is geworden, dan kan je zonder twijfel de CMSIS structs en pointers ook begrijpen…

```
675  #define RCC_BASE              (AHBPERIPH_BASE + 0x00001000)

1145 #define RCC                     ((RCC_TypeDef *) RCC_BASE)
```

```
456 typedef struct
457 {
458   __IO uint32_t CR;
459   __IO uint32_t CFGR;
460   __IO uint32_t CIR;
461   __IO uint32_t APB2RSTR;
462   __IO uint32_t APB1RSTR;
463   __IO uint32_t AHBENR;
464   __IO uint32_t APB2ENR;
465   __IO uint32_t APB1ENR;
466   __IO uint32_t BDCR;
467   __IO uint32_t CSR;
468   __IO uint32_t AHBRSTR;
469   __IO uint32_t CFGR2;
470   __IO uint32_t CFGR3;
471   __IO uint32_t CR2;
472 }RCC_TypeDef;
```

```
70 void InitIo(void)
71 {
72   // ------------------------------------------------------- //
73   // Info over de registers, zie: stm32f091xc.h (CMSIS code).
74
75   // Clock voor GPIOA inschakelen (on board LED).
76   RCC->AHBENR = RCC->AHBENR | RCC_AHBENR_GPIOAEN;
```

OPM: op lijn 1145 gaan we dus een 32-bit getal **casten** (omzetten) naar een *RCC_TypeDef* pointer en definiëren als *RCC*.

Kan je bovenstaande stukken code **zelf ook opzoeken** in je projectjes**?**

Probeer toch maar ;-).

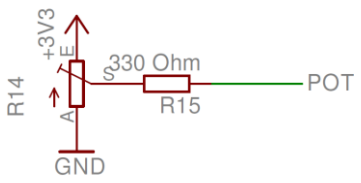
r1065698 Tim Kindt

AD-converter

Wat is een AD-converter?

Een AD-converter is een **Analoog-naar-Digitaal-omzetter**. Zo'n module kan dus een willekeurige analoge spanning vertalen naar een evenwaardig digitaal getal. De spanning op de pinnen van de microcontroller mag uiteraard niet boven zijn 'absolute maximum ratings' gaan.

Om onze AD-converter gemakkelijk te kunnen testen staat er op het Nucleo Extension Shield een trimmer geplaatst. Die trimmer kan je gebruiken om spanningen tussen 0 en 3V3 te bekomen. De uitgang van de trimmer is aangesloten op GPIOA pin 0.



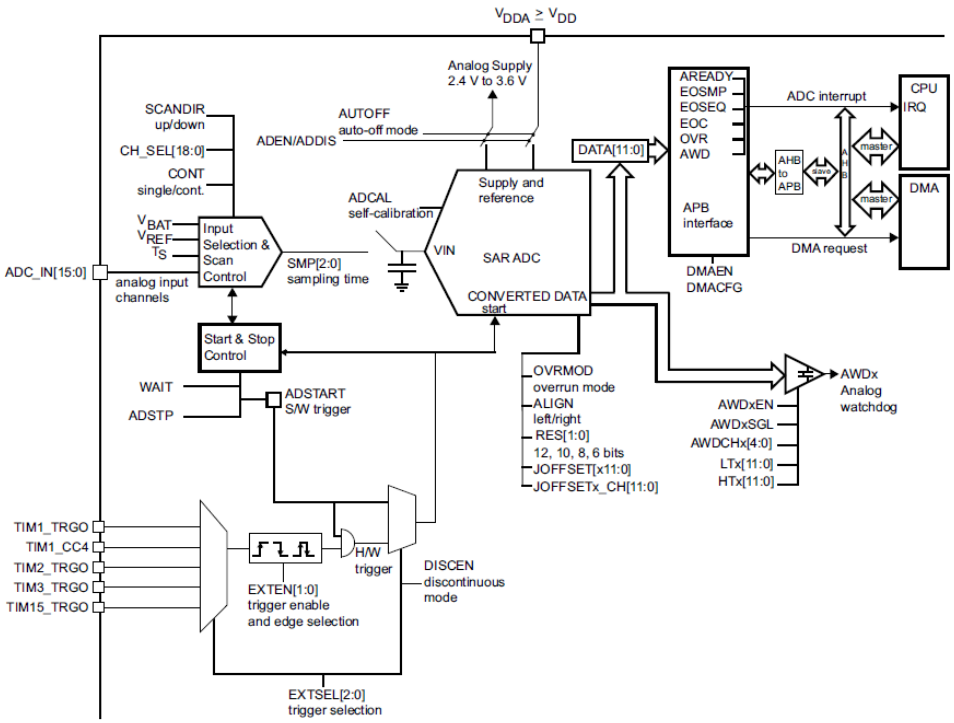
Figuur 31: trimmer op het Nucleo Extension Shield

De AD-converter van de STM32F091RC is een **12-bit** converter. De spanningen tussen 0 en 3V3 worden dus opgesplitst in 4096 gelijke stukjes…

Hoeveel Volt stelt één bitwaarde dan voor?

1 bit = …… mV

Het blokschema van de AD-converter ziet er als volgt uit:



Figuur 32: blokschema AD-converter (bron: RM0091)

Kan je het pad volgen van input tot op de databus?

r1065698 Tim Kindt

Instellingen voor het gebruik van een AD-converter

Om de AD-converter te kunnen gebruiken moeten een aantal instellingen correct gebeuren. Een aantal heel belangrijke zijn:

1. Plaats pin A0 op analoge mode
2. Voorzie een klok voor de AD-module
3. Kies de uitlijning
4. AD inschakelen
5. Kies een kanaal om te bemeten
6. Eventueel sample rate kiezen

Optioneel kan je ook nog andere zaken instellen:

1. Kalibratie uitvoeren
2. Auto turn off
3. Interrupts genereren
4. …

In code wordt dit:

```
// AD-module initialiseren
void InitAd(void)
{
  // AD-module van klok voorzien
  RCC->APB2ENR |= RCC_APB2ENR_ADC1EN;

  // right alignment (optioneel)
  ADC1->CFGR1 &= ~ADC_CFGR1_ALIGN;

  // AD enable'n
  ADC1->CR |= ADC_CR_ADEN;
  while ((ADC1->ISR & ADC_ISR_ADRDY) == 0);

  // AD-kanaal 0 kiezen (verbonden met Poort A0)
  ADC1->CHSELR = ADC_CHSELR_CHSEL0;

  // tragere sample rate kiezen (voor hoogimpedante analoge bronnen)
  ADC1->SMPR |= ADC_SMPR_SMP_0 | ADC_SMPR_SMP_1 | ADC_SMPR_SMP_2;
}
```

De AD-conversie starten en het resultaat uitlezen kan zo:

```
// De blokkerende versie van de AD-conversie (later met interrupts).
uint16_t GetAdValue(void)
{
  uint32_t temp = 0;

  ADC1->CR |= ADC_CR_ADSTART;                  /* Start the ADC conversion */
  while ((ADC1->ISR & ADC_ISR_EOSEQ) == 0);    // wachten op einde sequentie
  temp = ADC1->DR;

  return (uint16_t)temp;
}
```

Hoe zou je deze 12-bit waarde best kunnen tonen op de 8 LED's van het Nucleo Extension Shield?

r1065698 Tim Kindt

Timers

Wat is een hardware timer?

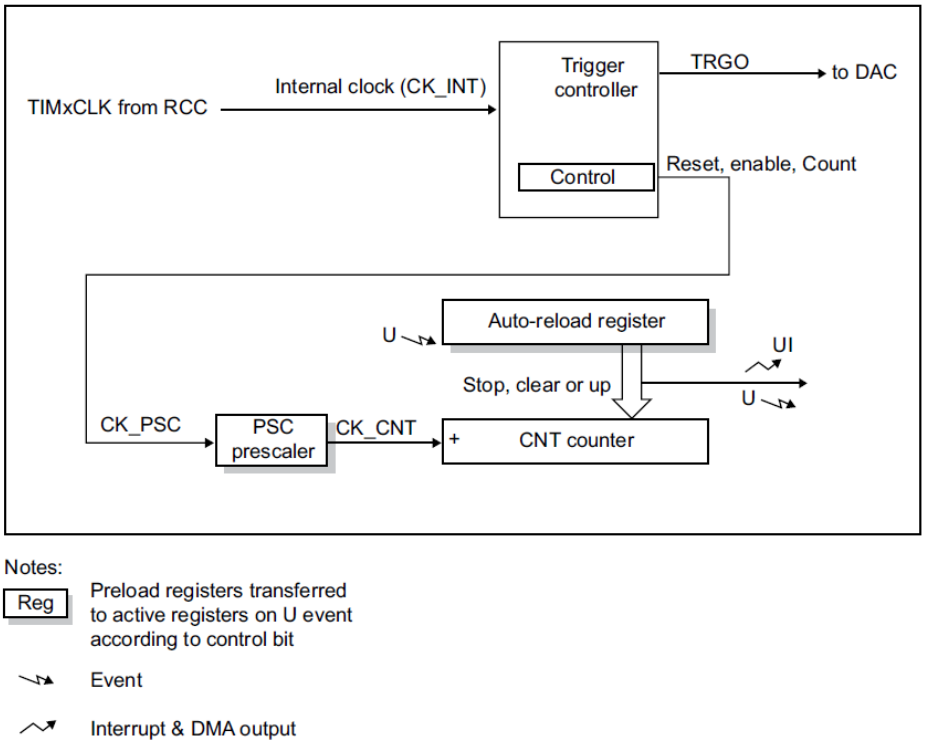
Een hardware timer is een digitale teller die **hardwarematig** (met flipflops) ingebed zit in de microcontroller. Het voordeel van een hardware timer is dat die timer **automatisch** kan **tellen**, **zonder** dat hij daarvoor de **tussenkomst van een CPU** nodig heeft.

Dit zorgt er dan weer voor dat je met hardware timers heel precies tijden kan gaan meten of instellen en de CPU nuttigere zaken te laten doen. De enige bepalende factor qua nauwkeurigheid is de stabiliteit van de oscillator waarmee de timer gevoed wordt.

Instellingen voor het gebruik van Timer 6/7

De STM32F091RC heeft 12 verschillende timers! Dit is behoorlijk wat.

Timer 6 en 7 hebben een gelijkaardige opbouw. Het zijn 16-bit auto-reload timers. Het principeschema vind je hieronder:



Figuur 33: blokschema Timer 6/7 (bron: RM0091)

Kan je de werking uitzoeken om een Update Interrupt (UI) te genereren vertrekkende van TIMxCLK?

r1065698 Tim Kindt

Toepassingsvoorbeeld: 1/8ste secondeklok

Hieronder de code om een interrupt te genereren om de 125ms. Die interrupt wordt gebruikt om LED8 en LED7 te laten knipperen.

```
  // Oneindig lang lopen...
  while (1)
  {
    // Niets doen...
  }
}
```

```
// Timer 6 instellen
void InitTimer6(void)
{
  RCC->APB1ENR |= RCC_APB1ENR_TIM6EN;    // clock voorzien voor de timer

  TIM6->PSC = 48000;                     // Als je een klok wil op
                                         // 1000 Hz = 48 000 000 / 48 000 = 1000 Hz...
  TIM6->ARR = 125;                       // Als je iedere 125 ms een interrupt wil.

  TIM6->DIER |= TIM_DIER_UIE;            // Interrupt enable voor timer 6
  TIM6->CR1 |= TIM_CR1_CEN;              // counter enable

  NVIC_SetPriority(TIM6_IRQn, 2);        // Kies een prioriteit...
  NVIC_EnableIRQ(TIM6_IRQn);             // Enable interrupt
}
```

```
// Interrupt van Timer 6 opvangen
void TIM6_IRQHandler(void)
{
  if((TIM6->SR & TIM_SR_UIF) == TIM_SR_UIF)
  {
    // Interruptvlag resetten
    TIM6->SR &= ~TIM_SR_UIF;

    // LED8 en LED7 toggle'n als bewijs van interrupt.
    GPIOB->ODR = GPIOB->ODR ^ GPIO_ODR_6;
    if(GPIOB->ODR & GPIO_ODR_6)
      GPIOC->ODR = GPIOC->ODR & ~GPIO_ODR_7;
    else
      GPIOC->ODR = GPIOC->ODR | GPIO_ODR_7;
  }
}
```

r1065698 Tim Kindt

Toepassingsvoorbeeld: PWM-generatie servomotor

Een **modelbouw servomotor** kan aangestuurd worden met een **PWM**-signaal. De totale periodetijd moet ongeveer **20ms** zijn. De **minimum** aan-tijd is **1ms**. De **maximum** aan-tijd is **2ms**. De tussenliggende tijden worden dan omgezet naar een servopositie.

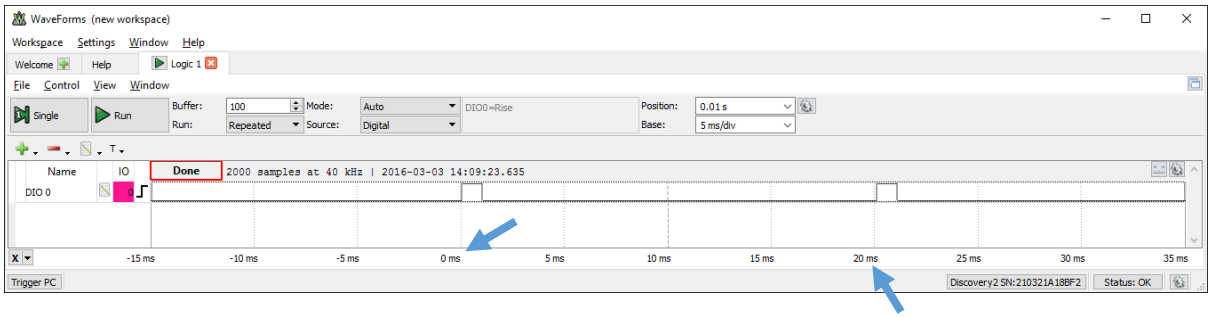
Figuur 34: modelbouw RC servo

Dergelijk signaal kan perfect gemaakt worden met een Timer module. Bedenkt zelf hoe je dit zou kunnen realiseren. In het ideale geval, doe je dat met een timer module in PWM-mode…

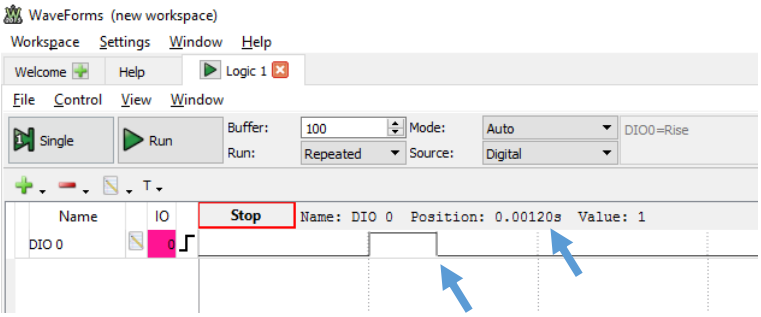
Oscilloscoopbeelden

Je kan scoopbeelden verwachten zoals hieronder aangegeven.

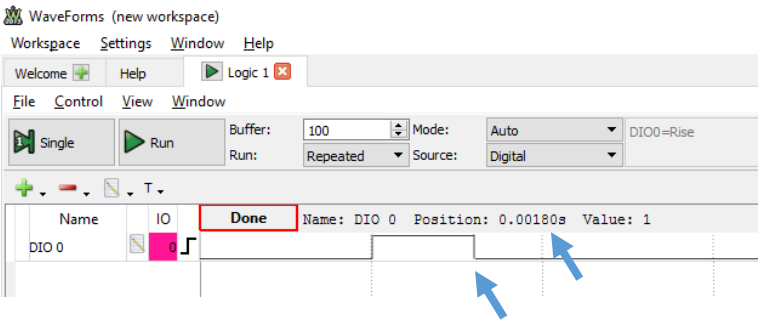
Scoopbeeld van een PWM-signaal voor een modelbouw RC servo. Bekijk de periodetijd. Die moet ongeveer 20ms zijn…



Scoopbeeld met een aan-tijd van 1,2ms.



Scoopbeeld met een aan-tijd van 1,8ms.



r1065698 Tim Kindt

Toepassingsvoorbeeld: HC-SR04 ultrasonic ranger

Met behulp van ultrageluid kan je een afstand sensor maken. Een HC-SR04 is zo'n sensor. De meting wordt **gestart** met een trigger puls (**trigger**) en van de **terugkomende puls** (**echo**) moet de lengte gemeten worden. **Die lengte is een maat voor de afstand tussen sensor en object.**

Een deel van de code vind je hieronder:

```
34    while (1)
35    {
36      // Trigger starten voor de HC-SR04
37      GPIOA->ODR |= GPIO_ODR_10;     //trigger op 1 zetten
38      WaitForMs(1);
39      GPIOA->ODR &= ~GPIO_ODR_10;    //trigger op 0 zetten
40
41      // Wachten tot echo terug is...
42      WaitForMs(150);
43
44      // Beetje herschalen en op LED's zetten.
45      ByteToLevel(distance*2);
46    }
```

```
49  // Timer 6 instellen
50  // 1cm (moet dubbel afgelegd worden) = 0.02m/340m/s = ~59 µs
51  // Gemeten met een klok van 48MHz is dit 59µs/20,8ns = ~2832 stappen
52  void InitTimer6(void)
53  {
54    RCC->APB1ENR |= RCC_APB1ENR_TIM6EN;   // clock voorzien voor de timer
55
56    TIM6->PSC = 0;                        // prescaler op nul, dus de timer telt op ritme van 48MHz
57    TIM6->ARR = 2832;                     // na 2832 stappen is er 1 cm afstand gemeten (heen en terug)
58
59    TIM6->DIER |= TIM_DIER_UIE;           // Interrupt enable voor timer 6
60
61    NVIC_SetPriority(TIM6_IRQn, 0);       // Kies een prioriteit...
62    NVIC_EnableIRQ(TIM6_IRQn);            // Enable interrupt
63  }
64
65  void Timer6On(void)
66  {
67    TIM6->CNT = 0;
68    TIM6->CR1 |= TIM_CR1_CEN;             // counter enable
69  }
70
71  void Timer6Off(void)
72  {
73    TIM6->CR1 &= ~TIM_CR1_CEN;            // counter disable
74  }
```

OPM: eigenlijk is het beter in 'Input Capture' mode te werken in dit geval. Bijvoorbeeld met Timer1. Daarmee kan je nog iets vlotter de pulstijd opmeten.

Toepassingsvoorbeeld: IR-communicatie met de IRTIM (Timer 17 en UART)

Als uitbreiding i.v.m. timers kan je zeker eens de IRTIM bekijken. Het moet je in staat stellen via **infrarood** te communiceren…

r1065698 Tim Kindt

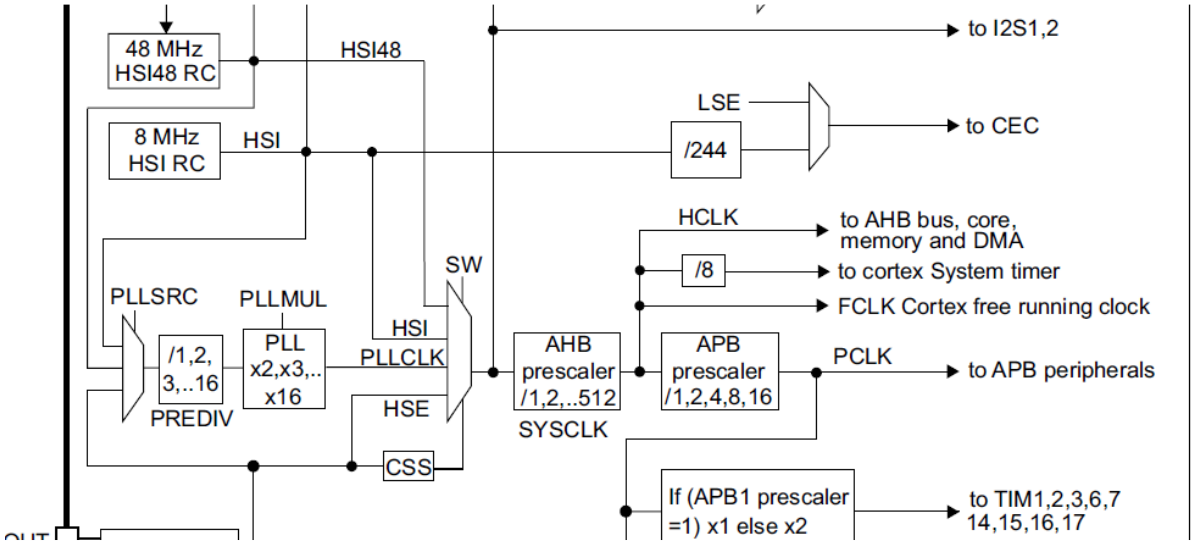
SysTick

Basisinstellingen

De SysTick timer is een timer die veelvuldig gebruikt wordt bij RTOS systemen (Real Time Operating System). Je kan instellen na hoeveel tijd er een 'tick' gegeven wordt en zo je RTOS laten switchen tussen allerhande taken.

De SysTick kan echter ook handig gebruikt worden als je geen RTOS draait, maar een 'gewoon' stuk embedded software.

De SysTick is een 24-bit **afteller** die aangestuurd wordt op ritme van HCLK of HCLK/8 (afhankelijk van type processor). Er is een interrupt bij het bereiken van 0…!



Figuur 35: gedeelte van de clock tree (bron: RM0091)

Een instelling om iedere 20 ms een tick te geven doe je zo:

```
SystemCoreClockUpdate();   // Nieuwe waarde van de core frequentie opslaan in
                           // SystemCoreClock variabele.

SysTick_Config(960000);    // Interrupt genereren. Zie core_cm0.h, om na iedere
                           // 20ms een interrupt te hebben op SysTick_Handler().
                           // 48 000 000 / 960 000 = 50 keer per seconde => 20ms.
                           // Let op: SysTick is 24-bit breed, dus maximum waarde
                           // is 16 777 215.
```

In de interrupt handler kan je dan bijvoorbeeld een LED toggle'n.

```
// Handler die iedere 20ms afloopt.
// Ingesteld met SystemCoreClockUpdate() en SysTick_Config().
void SysTick_Handler(void)
{
  // LED1 toggle'n
  GPIOC->ODR = GPIOC->ODR ^ GPIO_ODR_0;
}
```


r1065698 Tim Kindt

Tijd meten met SysTick

In sommige gevallen, bijvoorbeeld in regeltechnische systemen, is het van groot belang dat er rekening gehouden wordt met de tijdsduur die een bepaalde actie in beslag nam. Een SysTick timer kan daarbij heel handig zijn.

Onderstaand voorbeeld **meet** via de SysTick **hoe lang** de functie `DoSomeStuff()` **duurt**… Het resultaat is te zien op de seriële poort.

```
42  // SysTick uitschakelen
43  SysTick->CTRL = 0;
44
45  // 24-bit 'reload' getal op maximum zetten zodat je
46  // aftelt van maximum waarde na bereiken van 0.
47  SysTick->LOAD = 0xFFFFFF;
48
49  // Huidige waarde op nul zetten zodat straks onmiddellijk
50  // gereset wordt op 0xFFFFFF.
51  SysTick->VAL = 0;
52
53  // HCLK selecteren en SysTick Timer enable (geen interrupt toelaten).
54  SysTick->CTRL = SysTick_CTRL_CLKSOURCE_Msk | SysTick_CTRL_ENABLE_Msk;
55
56  // Wachten om herladen van de teller (met 0xFFFFFF)
57  while(SysTick->VAL == 0);
58
59  // Begintijd vaststellen
60  startTime = SysTick->VAL;
61
62  // Beetje uitrusten
63  DoSomeStuff();
64
65  // Eindtijd vaststellen
66  endTime = SysTick->VAL;
68  // Controleren of er geen underflow/roll over geweest is (onder nul gezakt).
69  if((SysTick->CTRL & 0x10000) == 0)
70  {
71    duration = startTime - endTime;
72    durationMs = (float)duration / 48;
73    sprintf(info, "\r\nDe opgemeten tijdsduur van de functie 'DoSomeStuff()' is: %d klokcycli.", duration);
74    StringToUsart(info);
75    sprintf(info, "\r\nDit staat gelijk met: %.2f microseconden.", durationMs);
76    StringToUsart(info);
77  }
78  else
79  {
80    duration = 0;
81    sprintf(info, "\r\nDe opgemeten tijdsduur van de functie 'DoSomeStuff()' is onbekend (SysTick underflow).");
82    StringToUsart(info);
83  }
```

COM26 - PuTTY

```
Tijdsduur van een functie meten via de SysTick registers.
---------------------------------------------------------
De opgemeten tijdsduur van de functie 'DoSomeStuff()' is: 8452 klokcycli.
Dit staat gelijk met: 176.08 microseconden.
```

WaitForMs()

Nu je het principe van de SysTick timer kent, kan je ook je functie **WaitForMs**() aanpassen zodat je via de System Tick je wachtfunctie uitvoert… Zie opdrachten in de labocursus.

r1065698 Tim Kindt

Alternate functions

Wat is een alternate function?

Een alternate function is letterlijk 'een alternatieve functie' die voorzien wordt op een pin van de microcontroller.

Alle pinnen gedragen zich in principe als één van de onderstaande mogelijkheden:

1. Digitale ingang
2. Digitale uitgang
3. Alternatieve functie
4. Analoge ingang

De keuze uit die vier gedragingen, worden ingesteld via het **GPIOx_MODER** register (pin mode register). De instellingen in dat register kunnen als volgt zijn:

MODERy[1:0]: Port x configuration bits (y = 0..15)
These bits are written by software to configure the I/O mode.
00: Input mode (reset state)
01: General purpose output mode
10: Alternate function mode
11: Analog mode

Figuur 36: pin configuration bits in het mode register (bron: RM0091)

Eerder in deze cursus hebben we reeds kennis kunnen maken met digitale in- en uitgangen en analoge ingangen. Het enige wat nog rest is een pin een alternatieve functie geven (10) …

Neem er de datasheet van de STM32F091RC bij en zoek in tabel 13 (p 42) het volgende op:

A3	95	B3	61	A6	45	PB8	I/O	FTf		I2C1_SCL, CEC, TIM16_CH1, TSC_SYNC, CAN_RX	-

Figuur 37: Pin definitions (bron: datasheet STM32F091RC)

Daarop zien we dat pin 8 van poort B (PB8) een input/output (I/O) is die geklasseerd is als 5V-tolerante pin (FT). Die pin kan zich ook nog op een 'alternatieve manier' gedragen namelijk:

- Als klok van I²C-bus 1 (I2C1_SCL)
- Als pin voor de CEC-communicatie (CEC)
- Als pin gekoppeld met channel 1 van timer 16 (TIM16_CH1)
- Als Touch Sensing Controller pin (TSC_SYNC)
- Als CAN-bus receiver pin (CAN_RX)

Verderop in deze cursus, bij het onderdeel I²C, zal deze pin ingesteld moeten worden als I2C1_SCL…

Instellingen voor het bekomen van een alternate function op een IO

Als je op een pin het gedrag wil instellen op de alternative functie, moet je een aantal stappen doorlopen. Stel na het voorzien van een **klok** voor de poort, eerst het **MODER** register in, daarna de **AFR** registers (Alternate Function Register). Per poort (A, B, C,…) is er een **GPIOx_AFRL** en

r1065698 Tim Kindt

GPIOx_AFRH register (zie datasheet RM0091, p 169). Deze registers worden in de CMSIS code, voor bijvoorbeeld poort B, respectievelijk genoteerd als
GPIOB->AFR[0] en GPIOB->AFR[1].

Duid hieronder aan waar de 4 bits zich bevinden die de alternate function instellingen bevatten voor **PB8**.

9.4.9 GPIO alternate function low register (GPIOx_AFRL) (x = A..F)

Address offset: 0x20

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
AFR7[3:0]				AFR6[3:0]				AFR5[3:0]				AFR4[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
AFR3[3:0]				AFR2[3:0]				AFR1[3:0]				AFR0[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **AFRy[3:0]:** Alternate function selection for port x pin y (y = 0..7)
These bits are written by software to configure alternate function I/Os

AFRy selection:

0000: AF0
0001: AF1
0010: AF2
0011: AF3
0100: AF4
0101: AF5
0110: AF6
0111: AF7
1000: Reserved
1001: Reserved
1010: Reserved
1011: Reserved
1100: Reserved
1101: Reserved
1110: Reserved
1111: Reserved

Figuur 38: Alternate function registers (bron: RM0091)

9.4.10 GPIO alternate function high register (GPIOx_AFRH) (x = A..F)

Address offset: 0x24

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
AFR15[3:0]				AFR14[3:0]				AFR13[3:0]				AFR12[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
AFR11[3:0]				AFR10[3:0]				AFR9[3:0]				AFR8[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **AFRy[3:0]:** Alternate function selection for port x pin y (y = 8..15)
These bits are written by software to configure alternate function I/Os

AFRy selection:

0000: AF0
0001: AF1
0010: AF2
0011: AF3
0100: AF4
0101: AF5
0110: AF6
0111: AF7
1000: Reserved
1001: Reserved
1010: Reserved
1011: Reserved
1100: Reserved
1101: Reserved
1110: Reserved
1111: Reserved

Figuur 39: Alternate function registers (bron: RM0091)

r1065698 Tim Kindt

In de alternate function registers (zoals hierboven), komen dus 4 bits per pin. Met die 4 bits kan je een bepaalde functie selecteren op die pin. De beschrijving voor de alternate functions kan je in de device specific datasheet vinden. Voor poort B wordt dit:

Table 15. Alternate functions selected through GPIOB_AFR registers for port B

Pin name	AF0	AF1	AF2	AF3	AF4	AF5
PB0	EVENTOUT	TIM3_CH3	TIM1_CH2N	TSC_G3_IO2	USART3_CK	-
PB1	TIM14_CH1	TIM3_CH4	TIM1_CH3N	TSC_G3_IO3	USART3_RTS	-
PB2	-	-	-	TSC_G3_IO4	-	-
PB3	SPI1_SCK, I2S1_CK	EVENTOUT	TIM2_CH2	TSC_G5_IO1	USART5_TX	-
PB4	SPI1_MISO, I2S1_MCK	TIM3_CH1	EVENTOUT	TSC_G5_IO2	USART5_RX	TIM17_BKIN
PB5	SPI1_MOSI, I2S1_SD	TIM3_CH2	TIM16_BKIN	I2C1_SMBA	USART5_CK_RTS	-
PB6	USART1_TX	I2C1_SCL	TIM16_CH1N	TSC_G5_IO3	-	-
PB7	USART1_RX	I2C1_SDA	TIM17_CH1N	TSC_G5_IO4	USART4_CTS	-
PB8	CEC	I2C1_SCL	TIM16_CH1	TSC_SYNC	CAN_RX-	-
PB9	IR_OUT	I2C1_SDA	TIM17_CH1	EVENTOUT	CAN_TX-	SPI2_NSS, I2S2_WS
PB10	CEC	I2C2_SCL	TIM2_CH3	TSC_SYNC	USART3_TX	SPI2_SCK, I2S2_CK
PB11	EVENTOUT	I2C2_SDA	TIM2_CH4	TSC_G6_IO1	USART3_RX	-
PB12	SPI2_NSS, I2S2_WS	EVENTOUT	TIM1_BKIN	TSC_G6_IO2	USART3_CK	TIM15_BKIN
PB13	SPI2_SCK, I2S2_CK	-	TIM1_CH1N	TSC_G6_IO3	USART3_CTS	I2C2_SCL
PB14	SPI2_MISO, I2S2_MCK	TIM15_CH1	TIM1_CH2N	TSC_G6_IO4	USART3_RTS	I2C2_SDA
PB15	SPI2_MOSI, I2S2_SD	TIM15_CH2	TIM1_CH3N	TIM15_CH1N	-	-

Figuur 40: Alternate functions voor poort B (bron: datasheet STM32F091RC)

Vind je de aanduiding waarbij je op pin 8 van poort B I²C klok kan instellen**?**

r1065698 Tim Kindt

UART

Wat is een UART?

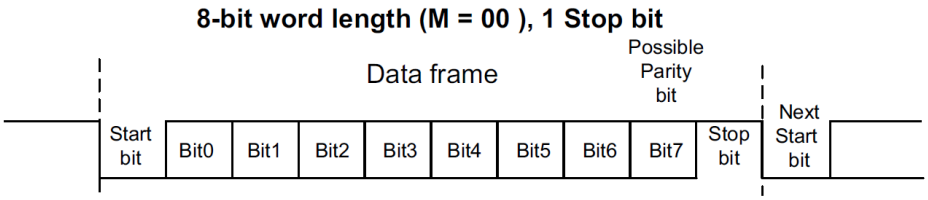
UART is de afkorting van **Universal Asynchronous Receiver** and **Transmitter**. Deze module kan gebruikt worden om bidirectioneel te communiceren tussen twee devices zonder een kloksignaal (vandaar asynchroon).

UART is wellicht beter gekend onder de vorm van RS-232, RS-485, RS-422, … De UART voorziet de correcte bitstroom, andere IC's kunnen dan de spanningsniveau's aanpassen zodat je kan spreken van bijvoorbeeld RS-232.

De UART module in de STM32F091RC is een 'speciaal geval'. Je kan er naast asynchrone communicatie ook synchrone communicatie mee verzorgen. Dan moet je wel een klok voorzien. Daarom spreekt men van USART (Universal Synchronous and Asynchronous Receiver and Transmitter). De synchrone versie valt echter buiten het bestek van deze cursus. We bekijken wel nog andere synchrone bussen zoals: I^2C en SPI.

UART data frame

De UART verstuurt quasi altijd pakketjes van 8 bit, bytes dus, één na één over een draadje. Eén zo'n byte/data frame ziet er zo uit:



Figuur 41: UART data frame (bron: RM0091)

Om 8 bit te versturen heb je dus 10 'tijdsloten' van één bit nodig. Er is namelijk een start- en een stopbit nodig. De start- en stopbit hebben als functie het synchroon laten lopen van de zender en ontvanger. Omdat we immers geen klok meesturen, moet er een mogelijkheid zijn voor de ontvanger om op een correcte manier de verzonden bits te ontvangen. Dat kan dus door gebruik te maken van twee principes:

1. Zender en ontvanger moeten op voorhand een **datatransfertsnelheid** afspreken. Hoe snel zal de data aankomen? Dat wordt typisch uitgedrukt in een aantal **baud**… Genaamd naar Émile Baudot die leefde in Frankrijk in de tweede helft van de 19de eeuw.
2. Via een **start- en stopbit** kan de ontvanger het begin en einde van een ontvangen byte detecteren.

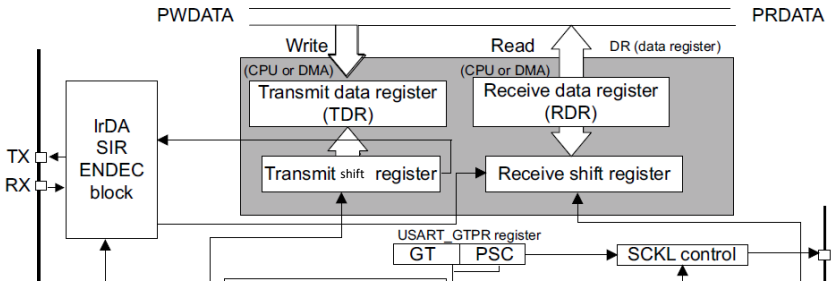
Nucleo en UART

Het Nucleo bordje beschikt over een on board ST-Link programmer die verbonden is met pinnen PA2 en PA3 van de STM32F091RC. Die pinnen kan je koppelen met UART2 en dus via USB. Daarmee kan je eenvoudig via een **virtuele COM-poort communiceren** met je computer…!

r1065698 Tim Kindt

Blokschema

Een gedeelte van het blokschema van de USART van de STM32F091RC vind je hieronder. Bekijk zeker ook de versie in de datasheet…



Figuur 42: gedeeltelijk blokschema USART (bron: RM0091)

Het register TDR wordt gebruikt om data te verzenden. RDR om data te ontvangen. De module waarin IrDA, SIR, enzovoort te zien is, zullen we voorlopig niet gebruiken.

Instellingen voor het gebruik van UART's

Gebruik van de UART is een alternatieve functie van een pin. Dus om de UART te gebruiken moet je verschillende stappen doorlopen:

1. De gebruikte poort van klok voorzien
2. De gebruikte pinnen op alternate function zetten
3. De alternate function kiezen
4. De UART initialiseren met de gewenste parameters (baud rate, aantal stopbits, …)

De parameters om enkel de transmitter in te stellen zijn:

```
// Usart module van een klok voorzien
RCC->APB1ENR |= RCC_APB1ENR_USART2EN;

// Baudrate zetten
USART2->BRR = 48000000/9600;

// USART inschakelen
USART2->CR1 |= USART_CR1_UE;

// Transmitter enable
USART2->CR1 |= USART_CR1_TE;
```

Complexiteit versus efficiëntie

Zowel voor het versturen als voor het ontvangen van data heb je de keuze tussen het werken via:

1. Polling
2. Interrupt gestuurd
3. DMA

De bovenstaande opsomming staat in volgorde van complexiteit. Polling (tijd gebaseerd) is de eenvoudigste vorm, met als nadeel dat de CPU dikwijls staat te 'wachten' op feedback van de UART-module. Of dat data verloren gaat.

DMA is de meest efficiënte manier van werken (in sommige gevallen), maar is relatief complex.

r1065698 Tim Kindt

Toepassingsvoorbeeld: communicatie op de virtuele COM-poort via polling

Als toepassingsvoorbeeld een stuk code dat data verzendt en ontvangt via USART2 (USB COM-poort) via 'polling'.

```
while (1)
{
  // Beetje wachten.
  for(ticks = 0; ticks < 3000000; ticks++)
    __NOP();

  // USER LED Nulceo bord toggle'n
  GPIOA->ODR = GPIOA->ODR ^ GPIO_ODR_5;

  // Teken van leven geven naar je computer.
  StringToUsart("\r\nHallo, ik leef nog. Je kent me wel, de STM32F091RC... ;-)");

  // Is er een byte ontvangen?
  // Is het 'Read data register not empty'?
  if((USART2->ISR & USART_ISR_RXNE) == USART_ISR_RXNE)
  {
    receivedByte = (uint8_t)USART2->RDR;
    ByteToLeds(receivedByte);
  }
}
```

Bovenstaande code is heel **eenvoudig**. Hét grote probleem echter is het ontvangen van data. De ontvangende microcontroller zal slechts één maal per lusdoorgang controleren of er een byte ontvangen is. Is dat het geval, komt de waarde van die byte op de LED's. Als er meer dan één byte aangekomen is tijdens deze laatste lusdoorgang, ben je die **voorgaande bytes kwijt**… Enkel de laatste byte blijft in het buffer register (RDR) en kan je opvragen.

r1065698 Tim Kindt

Toepassingsvoorbeeld: communicatie op de virtuele COM-poort via interrupts

Als toepassingsvoorbeeld een stuk code dat data verzendt en ontvangt via USART2 (USB COM-poort) via 'interrupts'. Merk op, enkel het ontvangen gebeurt interrupt gestuurd.

```
  // Koppeling van interrupt maken met de NVIC
  NVIC_SetPriority(USART2_IRQn, 2);
  NVIC_EnableIRQ(USART2_IRQn);
}

// Interrupt handler van USART2
void USART2_IRQHandler(void)
{
  // Byte ontvangen?
  if((USART2->ISR & USART_ISR_RXNE) == USART_ISR_RXNE)
  {
    // Byte ontvangen, lees hem om alle vlaggen te wissen.
    uint8_t temp = USART2->RDR;
    ByteToLeds(temp);
  }
}
```

Door interrupt gestuurd te werken, is de kans dat je inkomende bytes aan data verliest bijzonder gering geworden…

Toepassingsvoorbeeld: DMX genereren (uitbreiding)

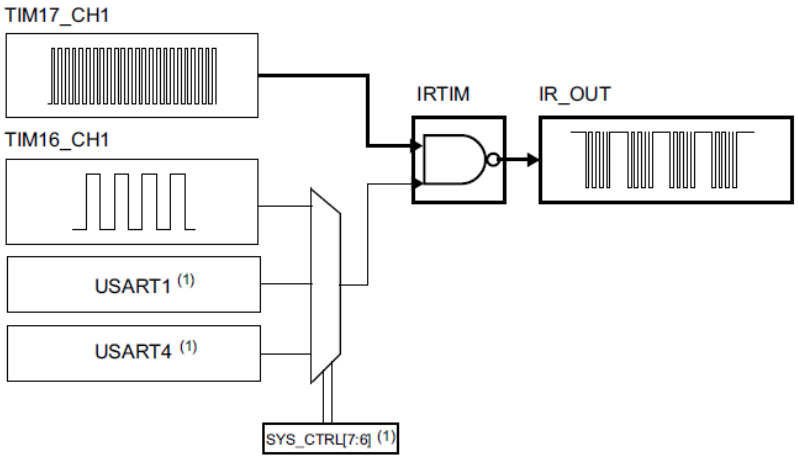
Als uitbreiding kan je nu wellicht zelf software maken die een DMX-console emuleert…?!

Toepassingsvoorbeeld: UART data versturen en ontvangen via infrarood

De standaard UART-signalen (TX en RX) kunnen omgezet worden naar signalen die via infrarood verstuurd en ontvangen kunnen worden. Er zijn verschillende manieren om dat te doen. Eén daarvan is het moduleren van de het TX-signaal op een (relatief) hoogfrequente drager. In ons voorbeeld gaan we het TX-signaal moduleren op 38kHz.

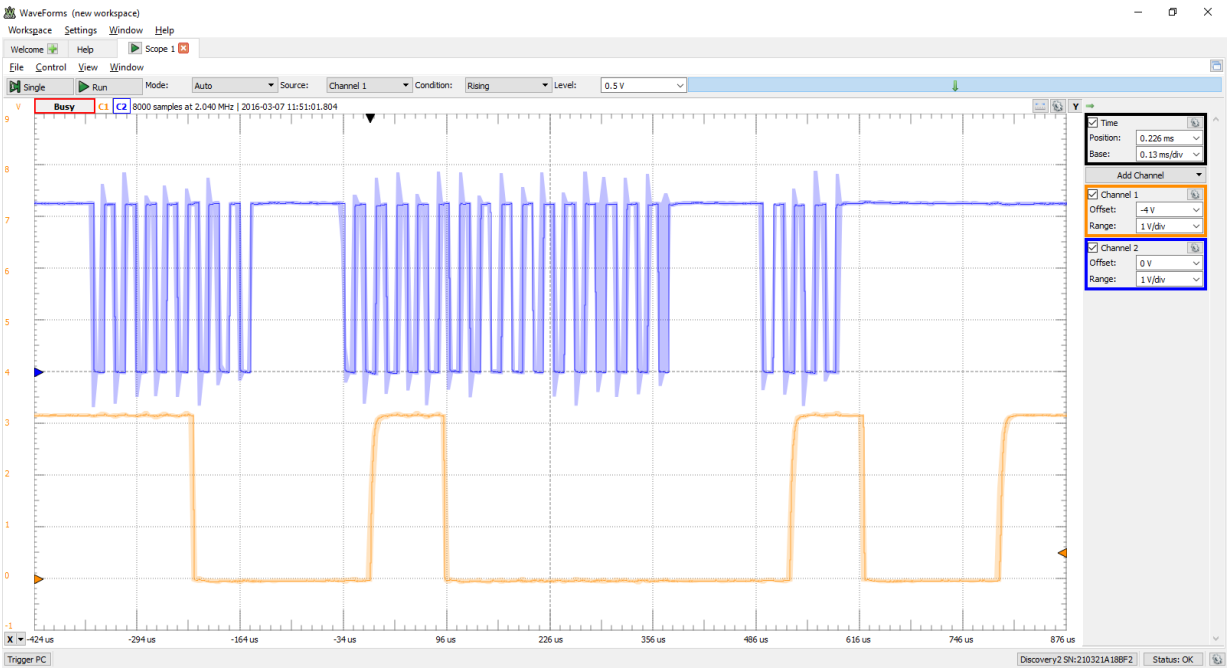
Het moduleren gebeurt in de microcontroller met behulp van de IRTIM-module. De IRTIM-module gebruikt PWM-channel 1 van timer 17 gecombineerd met USART1 om de communicatie op te bouwen.

De principiële opbouw zie je hieronder:



Figuur 43: IRTIM-module blokschema (bron: RM0091)

De code voor het realiseren van dit gemoduleerde signaal (zie volgende afbeelding) valt buiten het bestek van deze cursus. Het ontvangen met behulp van een demodulator (TSOP38338) is wel onderdeel van de cursus en volledig gelijklopend aan de hoger vermelde voorbeelden.



Figuur 44: Scoopafbeelding van het verzonden en ontvangen IR-signaal.

r1065698 Tim Kindt

SPI

Wat is SPI?

SPI is de afkorting van **Serial Peripheral Interface**. Het is een communicatiebus die **serieel** data kan doorsturen op een **synchrone** manier. Er is dus een klok voorzien (vandaar synchroon).

De SPI-bus wordt heel dikwijls gebruikt om op eenzelfde print (PCB) verschillende IC's met elkaar te koppelen. Voorbeelden zijn:

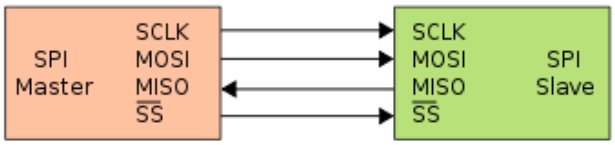
- microcontroller koppelen met een IO-expander
- microcontroller koppelen met een sensor
- microcontroller koppelen met een sensor, een actuator, een geheugenmodule,…

Bij SPI kan de kloksnelheid (en dus ook de datatransfertsnelheid) ingesteld worden. Hierbij is de maximumgrens onder andere bepaald door de slew rate van alle componenten op de bus, parasitaire capaciteiten, mogelijkheid tot instralen en opnemen van storingen,…

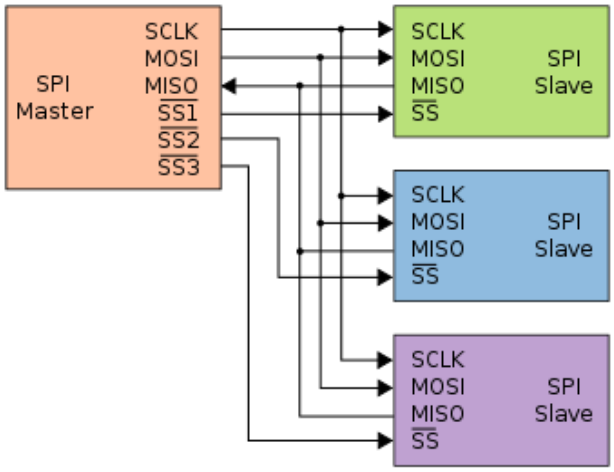
Kloksnelheden van enkele megahertzen zijn niet ongewoon.

Topologie

Er zijn verschillende manieren op een SPI-bus op te bouwen. Het kan zijn dat je slechts twee modules wil koppelen. Maar ook het koppelen van meer dan twee modules is mogelijk. Hieronder enkele voorbeelden.



Figuur 45: SPI single master/single slave (bron: wikipedia.org)



Figuur 46: SPI single master/mutiple slave (bron: wikipedia.org)

r1065698 Tim Kindt

SPI IO's

Om met de SPI-bus te werken worden meestal 2 tot 4 pinnen ingesteld.

1. **SCK**: serial clock, bepaalt de kloksnelheid.
2. **MOSI**: Master Output Slave Input, data uitgang van de master en ingang van de slave.
3. **MISO**: Master Input Slave Output, data ingang van de master en uitgang van de slave.
4. **$\overline{\mathbf{SS}}$:** Slave Select, pin die actief laag gemaakt moet worden om de slave te activeren.

Pinnen SCK en MOSI zijn essentieel, MISO en $\overline{\mathbf{SS}}$ zijn optioneel.

Instellingen voor het gebruik van SPI

Om met de SPI-module te kunnen werken in de STM32F091RC moeten volgende stappen doorlopen worden:

1. Gebruikte poorten van klok voorzien.
2. Gebruikte SPI-module van klok voorzien.
3. Alternate functions instellen voor de SPI-pinnen.
4. Gewenste kloksnelheid instellen.
5. Software slave management inschakelen indien je geen slave select pin gebruikt.
6. Master mode instellen.
7. FIFO reception threshold op ¼ te zetten (8-bit trigger).
8. SPI inschakelen.

Optioneel zijn:

1. Klokpolarisatie instellen.
2. Klokfase instellen.
3. Bidirectionele mode instellen.
4. LSB of MSB first instellen.
5. Internal slave select instellen.
6. Aantal databits instellen.
7. …

```
void InitSpi1(void)
{
    // Poort A van klok voorzien (mocht dit nog niet gebeurd zijn).
    RCC->AHBENR = RCC->AHBENR | RCC_AHBENR_GPIOAEN;

    // RCC_APBxENR registers instellen zodat de SPI-modules een klok krijgt.
    RCC->APB2ENR = RCC->APB2ENR | RCC_APB2ENR_SPI1EN;       //SPI1 enable

    // SPI1 moet op de alternate function pinnen komen
    GPIOA->MODER = (GPIOA->MODER & ~(GPIO_MODER_MODER5)) | (GPIO_MODER_MODER5_1);
    GPIOA->MODER = (GPIOA->MODER & ~(GPIO_MODER_MODER6)) | (GPIO_MODER_MODER6_1);
    GPIOA->MODER = (GPIOA->MODER & ~(GPIO_MODER_MODER7)) | (GPIO_MODER_MODER7_1);
    GPIOA->AFR[0] &= 0x11011111;
    GPIOA->AFR[0] &= 0x10111111;
    GPIOA->AFR[0] &= 0x01111111;

    // SPI1 mode instellen
    SPI1->CR1 = SPI1->CR1 | SPI_CR1_BR_2 | SPI_CR1_BR_1 | SPI_CR1_BR_0;
    SPI1->CR1 = SPI1->CR1 & ~SPI_CR1_CPOL;
    SPI1->CR1 = SPI1->CR1 & ~SPI_CR1_CPHA;
    SPI1->CR1 = SPI1->CR1 & ~SPI_CR1_BIDIMODE;
    SPI1->CR1 = SPI1->CR1 & ~SPI_CR1_LSBFIRST;
    SPI1->CR1 |= SPI_CR1_SSM;
    SPI1->CR1 |= SPI_CR1_SSI;
    SPI1->CR1 |=  SPI_CR1_MSTR;
    SPI1->CR2 = (SPI1->CR2 & ~(SPI_CR2_DS)) | SPI_CR2_DS_0 | SPI_CR2_DS_1 | SPI_CR2_DS_2;
    SPI1->CR2 = SPI1->CR2 & ~SPI_CR2_FRF;
    SPI1->CR2 |= SPI_CR2_FRXTH;
    SPI1->CR1 |= SPI_CR1_SPE;
}
```

r1065698 Tim Kindt

Byte versturen

Een mogelijkheid is om via zelf geschreven functie een byte te verzenden via de SPI-bus.

Bekijk zelf eens de pointerconversies … Waarom staat de while-lus er…?

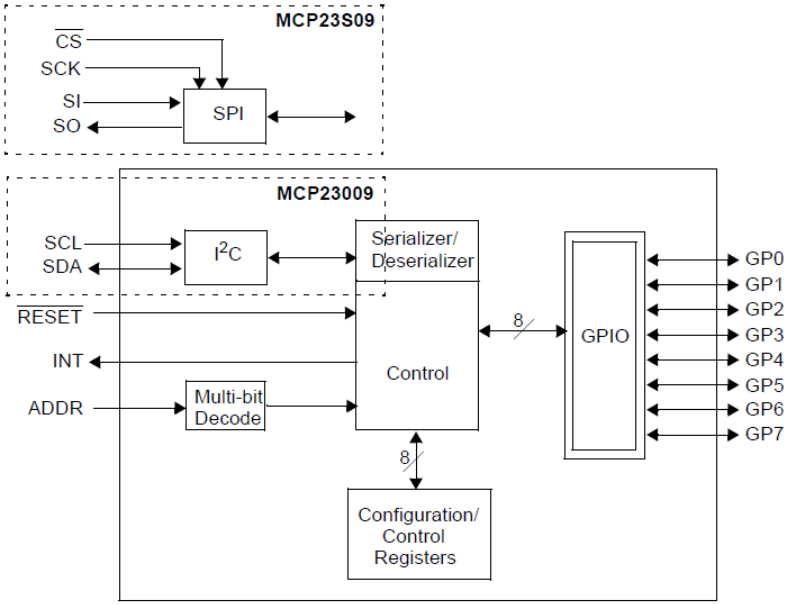
```
void ByteToSpi1(uint8_t data)
{
    // SPI1 data verzenden
    *(uint8_t *)&(SPI1->DR) = data;
    while((SPI1->SR & SPI_SR_BSY) == SPI_SR_BSY);
}
```

r1065698 Tim Kindt

Toepassingsvoorbeeld: IO-expander (lab CAD, MCP23S09)

Om meer inputs en outputs te voorzien op een bestaand systeem (microcontroller), kan je gebruik maken van een IO-expander. Zo'n IC kan via SPI (of I²C) de toestand van 8 pinnen instellen (output) of uitlezen (input).

Microchip is één fabrikant van dergelijk IC, namelijk: MCP23S09. Dat IC wordt ook gebruikt in de lessen Labo CAD…

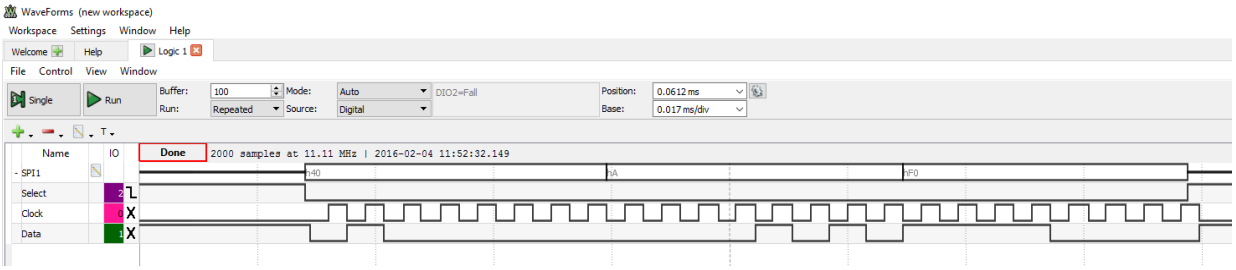


Kan je uw eigen IO-expander aan de praat krijgen met het Nucleo bord?

Hieronder een beeld van een meting met de oscilloscoop op de SPI-bus gekoppeld met een IO-expander.

Kan je **uitzoeken** wat de **toestand** van de **LED's** is?

Waarom worden 24 bits verstuurd?

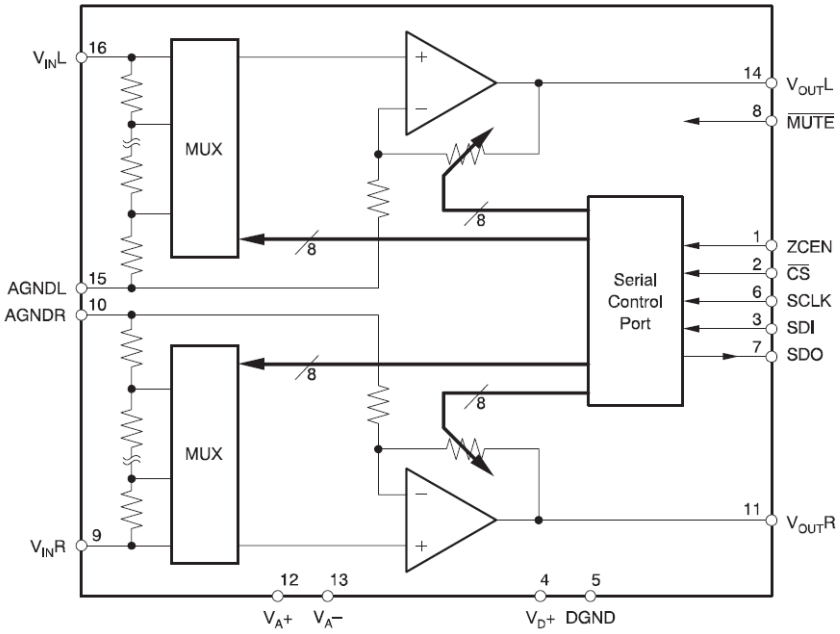


r1065698 Tim Kindt

Toepassingsvoorbeeld: PGA2311 volumeregeling

Ook volumes van audiosignalen kunnen versterkt of verzwakt worden via SPI-modules. Eén voorbeeld hiervan is de PGA2311.

Bekijk het blokschema. Kan je de SPI-interface terugvinden?



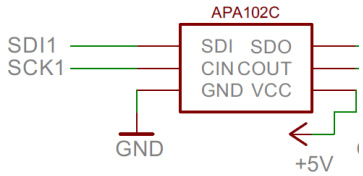
Figuur 47: Blokschema PGA2311 (bron: Burr Brown datasheet PGA2311).

r1065698 Tim Kindt

Toepassingsvoorbeeld: APA102C addressable LED

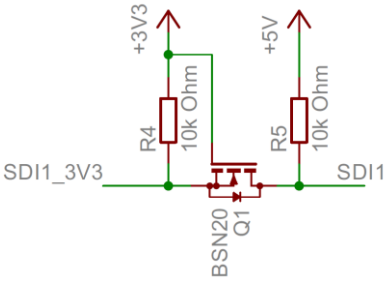
Meer en meer toepassingen met LED-strips verwachten adresseerbare LED's. Een recente ontwikkeling daarbij is het gebruik van RGB-LED's die via SPI aan gestuurd kunnen worden. Bijvooreeld de APA102C. De zogenaamde 'super LED'…

De aansluiting is als volgt:



Figuur 48: Aansluitingen APA102C (bron: rubu.be, super Wheel LED's mini).

De APA102C LED's zijn 5V gevoed. Ook hun SPI-bus moet dus 5V niveau's behandelen. Omdat onze microcontroller op 3V3 werkt, moet dus een **level converter** gebruikt worden. Een eenvoudige bidirectionele level converter kan je maken als volgt:



Figuur 49: Bidirectionele level converter met 1 MOSFET (bron: rubu.be, super Wheel LED's mini).

Kan je de **werking van bovenstaande schakeling** verklaren?

Codevoorbeeld:

```
while(1)
{
  // Willekeurig kleur kiezen tussen 1 en 7.
  kleur = rand()%7+1;
  monoChromeColumn.color = kleur;
  monoChromeColumn.ledStates = 0x0001;
  for(temp = 0; temp < 15; temp++)
  {
    DisplayMonoChromeColumn(&(monoChromeColumn));
    monoChromeColumn.ledStates <<= 1;
    WaitForMs(SPEED);
  }

  // Willekeurig kleur kiezen tussen 1 en 7.
  kleur = rand()%7+1;
  monoChromeColumn.color = kleur;
  monoChromeColumn.ledStates = 0x8000;
  for(temp = 0; temp < 15; temp++)
  {
    DisplayMonoChromeColumn(&(monoChromeColumn));
    monoChromeColumn.ledStates >>= 1;
    WaitForMs(SPEED);
  }
}
```


r1065698 Tim Kindt

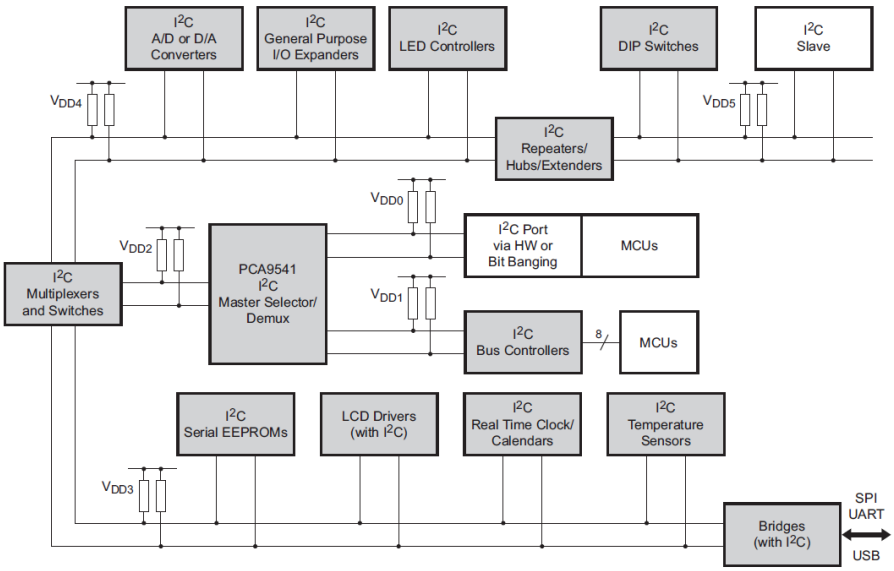
I²C

I²C is de afkorting van **Inter Integrated Circuit**. Het is een digitale multi-master multi-slave synchrone communicatiebus. Ze kan gebruikt worden om te communiceren tussen twee IC's. Dat kan op één en hetzelfde printbord zijn, maar ook op een apart bord. Gelijk in welke situatie, is I²C niet geschikt voor grote afstanden (best <1 m).

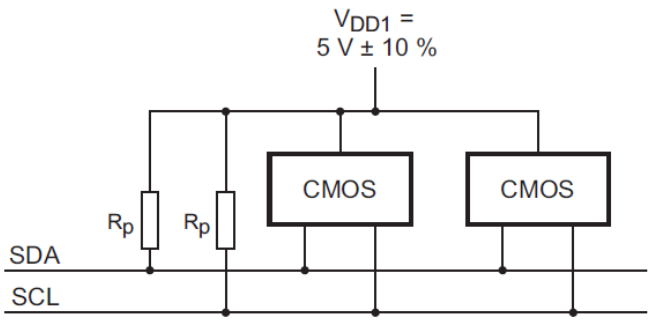
Om te communiceren zijn twee draadjes nodig: **SDA** (Serial Data) en **SCL** (Serial Clock). Deze signalen moeten via **pull-up** weerstanden naar de voedingsspanning getrokken worden. Hierbij moet uiteraard rekening gehouden worden met de voedingsspanning van alle devices. Uiteraard moeten alle devices een zelfde referentiespanning hebben (massa).

Het maximum aantal deelnemers is beperkt door de adres- en maximale **buscapaciteit** (400pF). De standaard gebruikte (laagste) klokfrequentie is 100kHz.

Een typische opbouw van een I²C-systeem is hieronder te zien. Let wel, het is een volledig overzicht van wat mogelijk is. In de meeste gevallen kan het schema eenvoudiger (2 devices met hun pull-up weerstanden). Daarvan een voorbeeld in op de volgende figuur.



Figuur 50: I²C-bustopologie (bron: Philips I²C bus specifications)



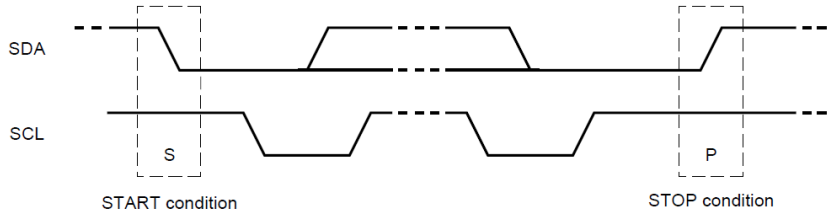
Figuur 51: Eenvoudige I²C-opstelling (bron: Philips I²C bus specifications)

Veel gebruikte pull-up weerstandswaardes zijn 4k7 Ohm.

Wat zou de reden zijn dat er met *pull-up* weerstanden gewerkt wordt?

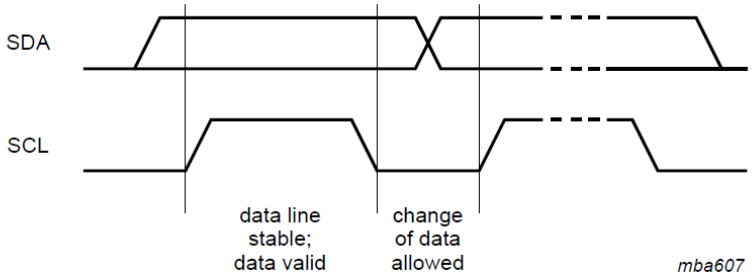
r1065698 Tim Kindt

Omdat I²C een bidirectionele bus is, moeten er afspraken gemaakt worden rond de manier van versturen van data. Eén van die afspraken is het werken met start- en stopcondities. Die zijn als volgt gedefinieerd:



Figuur 52: start- en stopcondities bij I²C (bron: Philips I²C bus specifications)

Daarmee samenhangend is er ook een afspraak rond de periode waarin data stabiel moet zijn of kan variëren.



Figuur 53: datacondities bij I²C (bron: Philips I²C bus specifications)

De I²C-module op onze ARM microcontroller zal al deze timing voorwaardes voor zijn rekening nemen (eenmaal de correct instellingen gedaan zijn). De STM32F091RC heeft 2 I²C-modules.

Instellingen voor het gebruik van I²C

Om pinnen als I²C-pinnen te gebruiken, moeten een aantal instellingen gedaan worden. Hieronder een summier overzicht:

1. Pinnen ingestellen als alternate function
2. I²C-module van klok voorzien
3. Timing instellen (100kHz, rise and fall times, …)
4. I²C-module inschakelen

Bij het versturen van data (byte(s)), moeten nog andere instellingen gebeuren:

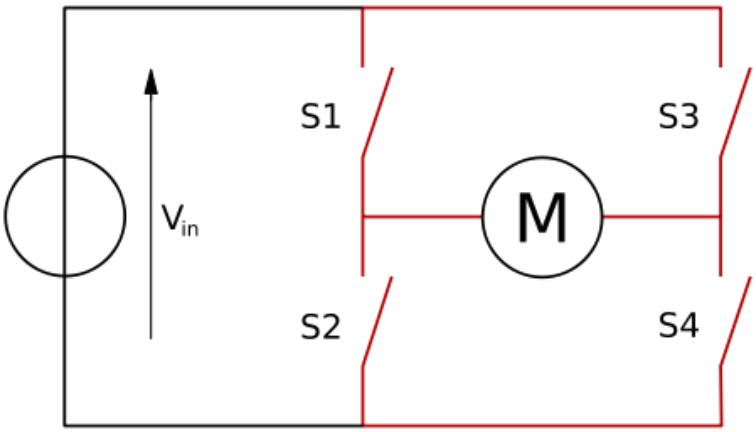
1. Kiezen tussen lezen en schrijven
2. Slave address goed zetten
3. Aantal te versturen/ontvangen bytes instellen
4. Startconditie verzenden
5. Byte klaarzetten (indien je wil verzenden)
6. Stopconditie verzenden

r1065698 Tim Kindt

Toepassingsvoorbeeld: I²C H-brug

Als toepassingsvoorbeeld het aansturen van een DC-motor met een I²C H-brug. De H-brug kan, door zijn specifieke opstelling, de stroom door een DC-motor van richting laten veranderen. Hierdoor zal de motor zowel links als rechts kunnen draaien.

Kan je verklaren hoe het links en rechts draaien werkt?



Figuur 54: principeopstelling van een H-brug (bron: wikipedia.org)

r1065698 Tim Kindt

De gebruikte H-brug kan werken op spanningen tot 16Vdc en stromen verwerken tot 10A. Ze verwacht via I²C twee bytes aan data: direction en speed. Het adres van de module is 0x41 (karakter 'A').

Een mogelijke initialisatie van de I²C-module verloopt als volgt:

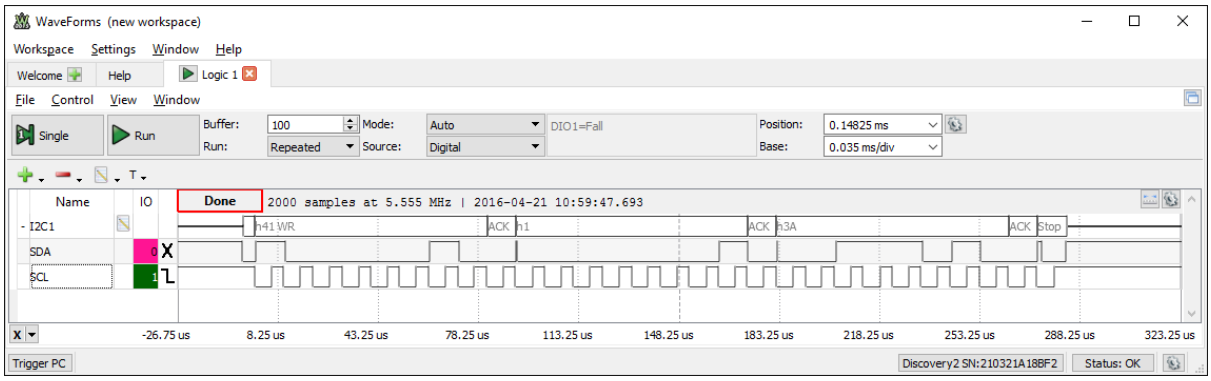
```
// Pinnen van de I²C1 instellen als alternate function.
// PB8 => SCL-pin (5V tolerant)
GPIOB->MODER = (GPIOB->MODER & ~GPIO_MODER_MODER8) | GPIO_MODER_MODER8_1;
GPIOB->AFR[1] |= 0x00000001;

// PB9 => SDA-pin (5V tolerant)
GPIOB->MODER = (GPIOB->MODER & ~GPIO_MODER_MODER9) | GPIO_MODER_MODER9_1;
GPIOB->AFR[1] |= 0x00000010;

// I²C1-module van een klok voorzien
RCC->APB1ENR |= RCC_APB1ENR_I2C1EN;

// Timing correct instellen (via AN4235 en Excel bestand van ST)
// 100kHz vertrekkende vanuit 8MHz (HSI)
I2C1->TIMINGR = 0x00201D2B;

// I²C1-module enable
I2C1->CR1 |= I2C_CR1_PE;
```

Het versturen van een nieuwe waarde kan dan als volgt:

```
// Nieuwe waarde versturen via I²C.
// Schrijfactie.
I2C1->CR2 &= ~I2C_CR2_RD_WRN;

// Slave address waarmee gecommuniceerd moet worden invullen.
I2C1->CR2 &= 0xFFFFFF00;                          // slave address resetten
I2C1->CR2 |= (SLAVE_ADDRESS << 1);                // adres klaarzetten

// Aantal te versturen bytes, waarna een stop gegenereerd moet worden (TC vlag wordt gezet).
I2C1->CR2 &= ~I2C_CR2_NBYTES;                     // NBYTES wissen
I2C1->CR2 |= (2 << 16);                           // 2 bytes te versturen, dus 2 in NBYTES plaatsen.

// Start conditie.
I2C1->CR2 |= I2C_CR2_START;

// Bytes versturen.
while((I2C1->ISR & I2C_ISR_TXE) != I2C_ISR_TXE);  // Is verzendbuffer leeg?
I2C1->TXDR = direction;                           // direction versturen
while((I2C1->ISR & I2C_ISR_TXE) != I2C_ISR_TXE);  // Is verzendbuffer leeg?
I2C1->TXDR = speed;                               // speed versturen

// Stop conditie.
while((I2C1->ISR & I2C_ISR_TC) != I2C_ISR_TC);    // Wacht op transmission complete.
I2C1->CR2 |= I2C_CR2_STOP;
```

Kan je zelf een doordachte signatuur bedenken voor een functie die data moet kunnen versturen op deze I²C-bus?

De communicatie kan ook gemonitord worden met een USB-oscilloscoop. Een scoopbeeld vind je hieronder:



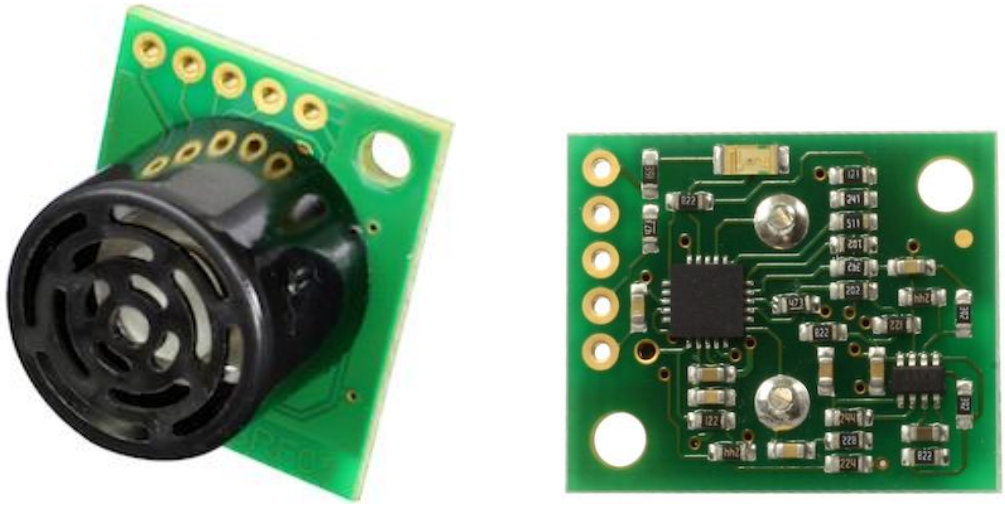
Merk telkens de 'aknowledge' op die de slave gebruikt om teken van leven te geven aan de master…

Wat is er in bovenstaand voorbeeld verstuurd geweest? Wat verwacht je dat de H-brug doet?

r1065698 Tim Kindt

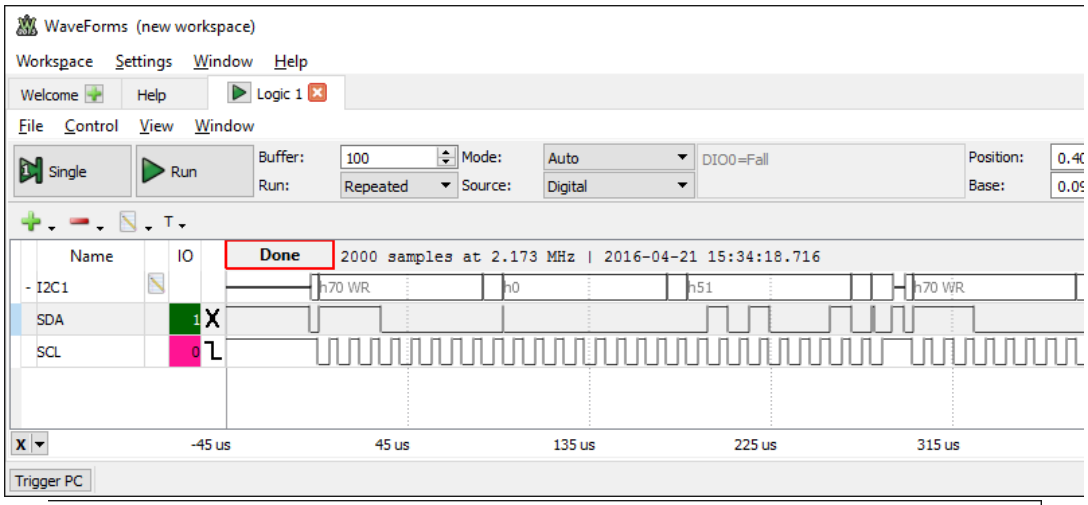
Toepassingsvoorbeeld: ultrasonic ranger (SRF-02)

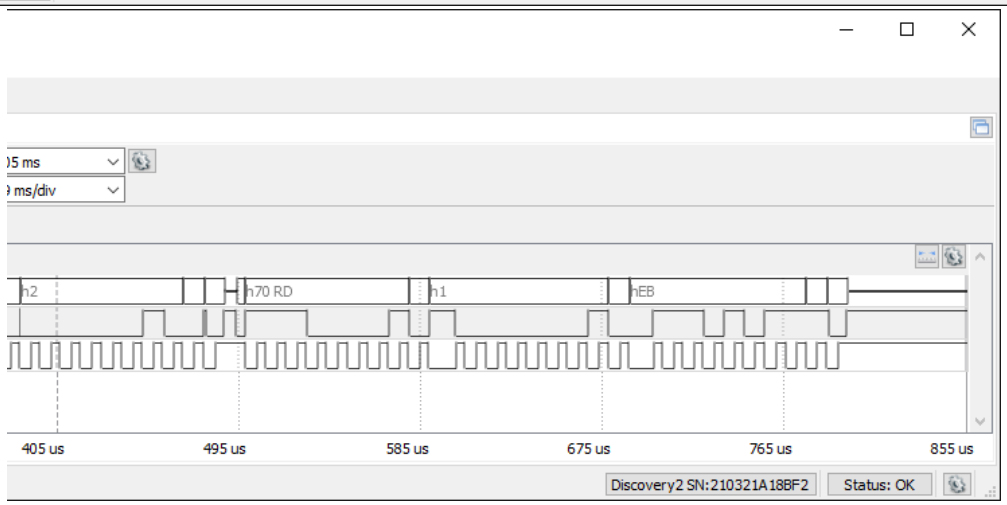
De SRF-02 module is een ultrasonic ranger (afstandsmeter) op I²C.



Figuur 55: http://www.robot-electronics.co.uk/htm/srf02techI2C.htm

Bekijk het oscilloscoopbeeld en zoek uit op welke afstand de sensor iets 'gezien' heeft…





r1065698 Tim Kindt